Cairo University
Faculty of Computing and Artificial Intelligence
Department of Computer Science

AutoPrepAI

Explainable Multi-Agent Data Preprocessing

Supervised by

Dr: Mohamed El-Ramly

TA: Amany Mohamed

Implemented by

Name	ID
Hassan Sherif	20220112
Zinab Mohamed	20220148
Nada Gamal	20220359
Maryam Hossam Eldin	20220328
Mahmoud Ayman	20220313

Graduation Project
Academic Year 2025-2026
Final Documentation

Table of Contents

Table of Tables

Table of Figures

Abstract

Data preprocessing is a critical but time-consuming step in the data analytics and machine learning pipeline. Many datasets suffer from missing values, outliers, inconsistencies, and redundant records, which negatively affect analysis quality and model performance. Existing data cleaning tools often require technical expertise or operate as black-box systems, limiting user understanding and control.

This project presents **AutoPrepAI**, an agentic AI-driven data cleaning and preprocessing system designed to provide an interactive, transparent, and user-friendly experience. The system enables users to upload datasets, detect data quality issues, and apply preprocessing operations through a chat-based interface with guided recommendations. By combining modern web technologies with machine learning techniques for data profiling and preprocessing, **AutoPrepAI** balances automation with explainability, allowing users to understand and control each data-cleaning step.

AutoPrepAI aims to reduce the time and effort required for data preparation while improving dataset quality and usability. The system supports both technical and non-technical users in producing cleaner, more reliable datasets, ultimately facilitating more accurate analytics and machine learning applications.

تُعد مرحلة معالجة البيانات المسبقة من المراحل الأساسية ولكنها تستغرق وقتًا وجهدًا كبيرين في مسار تحليل البيانات وتطبيقات التعلم الآلي. إذ تعاني العديد من مجموعات البيانات من القيم المفقودة، والقيم الشاذة، وعدم الاتساق، والسجلات المكررة، مما يؤثر سلبًا على جودة التحليل وأداء النماذج. كما أن أدوات تنظيف البيانات المتاحة حاليًا غالبًا ما تتطلب خبرة تقنية متخصصة أو تعمل كنظم مغلقة تحد من فهم المستخدم وتحكمه في عملية المعالجة.

يقدم هذا المشروع **AutoPrepAI**، وهو نظام ذكي لمعالجة وتنظيف البيانات يعتمد على مفهوم الوكلاء الذكيين (Agentic AI)، ويهدف إلى توفير تجربة تفاعلية وشفافة وسهلة الاستخدام. يتيح النظام للمستخدمين رفع مجموعات البيانات، واكتشاف مشكلات جودة البيانات، وتطبيق عمليات المعالجة المسبقة من خلال واجهة محادثة وتوصيات موجهة. ومن خلال دمج تقنيات الويب الحديثة مع أساليب التعلم الآلي الخاصة بوصف البيانات ومعالجتها، يحقق **AutoPrepAI** توازنًا بين الأتمتة وقابلية التفسير، مما يمكّن المستخدمين من فهم كل خطوة من خطوات التنظيف والتحكم فيها.

يهدف النظام إلى تقليل الوقت والجهد اللازمين لإعداد البيانات، وتحسين جودة البيانات وقابليتها للاستخدام، ودعم المستخدمين التقنيين وغير التقنيين في إنتاج مجموعات بيانات أكثر موثوقية، مما يساهم في الحصول على تحليلات أكثر دقة وتحسين أداء تطبيقات التعلم الآلي.

Chapter 1. Introduction

1.1 Background and Motivation

The rapid growth of data-driven applications has made data preprocessing one of the most critical stages in the machine learning lifecycle. Before a predictive model can be trained, raw datasets must be cleaned, transformed, and prepared to ensure high-quality input [1]. However, data preprocessing is often a complex, repetitive, and time-consuming process that requires significant domain knowledge [2]. Common issues such as missing values, duplicate records, outliers, inconsistent data types, and improperly formatted features can negatively affect model performance if not handled correctly [3].

Although several preprocessing tools exist, many of them require users to possess technical expertise and make manual decisions regarding suitable preprocessing techniques [3]. This creates a challenge for beginners and increases the time required for experienced data scientists. Recent advances in Artificial Intelligence (AI) and Large Language Models (LLMs) provide an opportunity to automate these tasks while offering intelligent guidance and explanations [4].

These challenges motivated the development of **AutoPrepAI**, an intelligent preprocessing platform that assists users in preparing datasets efficiently through AI-generated recommendations while keeping users involved in the decision-making process.

1.2 Problem Definition

Data preprocessing remains one of the most difficult and time-consuming stages of machine learning projects [1], [2]. Selecting appropriate preprocessing techniques often depends on user experience, leading to inconsistent decisions and increased development time [3], [4]. Existing automated preprocessing solutions frequently apply transformations without providing sufficient explanations or allowing user intervention, reducing transparency and trust [4], [5].

The problem addressed by this project is the lack of intelligent and explainable data preprocessing solutions that balance automation with human oversight. AutoPrepAI seeks to bridge this gap by providing an AI-driven preprocessing system that automates data preparation tasks while allowing users to review and approve preprocessing decisions prior to execution, thereby enhancing transparency, accountability, and user confidence.

1.3 Project Objectives and Proposed Solution

The primary goal of **AutoPrepAI** is to develop an intelligent, AI-driven platform that simplifies and accelerates data preprocessing activities. By combining automated preprocessing capabilities with human oversight, the platform ensures greater accuracy, transparency, and user trust in the generated results.

The project objectives are:

- Automatically detect common data quality issues.
- Explain the reasoning behind every action.
- Implement a Human-in-the-Loop workflow that allows users to approve, reject, or modify tool actions.
- Support multiple dataset formats, including CSV and Excel.
- Provide an AI assistant that answers preprocessing-related questions.
- Allow users to preview, track, and export processed datasets.

To achieve these objectives, **AutoPrepAI** integrates machine learning (ML) preprocessing techniques with Large Language Models (LLMs) to provide intelligent and explainable recommendations throughout the data preprocessing workflow. The system is implemented as an interactive web application utilizing React for the frontend, FastAPI for the backend, and PostgreSQL for data management and **Backblaze B2 Cloud** Storage for secure and scalable file storage. ensuring a scalable, efficient, and user-friendly platform.

1.4 Project Scope and Limitations

1.4.1 Project Scope

AutoPrepAI focuses on intelligent data preprocessing before machine learning model development. The system supports dataset upload, data profiling, feature transformations, dataset preview, operation history, and export of cleaned datasets. Users remain in control through Human-in-the-Loop validation, ensuring reliable preprocessing decisions.

1.4.2 Project Limitations

- Supports structured tabular datasets only.
- Supports English language only.
- Supports CSV and Excel file formats only.

- Dataset size is limited to a maximum of 5,000,000 cells (Rows × Columns ≤ 5,000,000). (e.g., 50000 rows and 100 columns OR vice versa)
- Performance depends on the quality of uploaded datasets.
- Currently designed for single-user usage without collaborative editing.

1.5 Development Methodology

The project follows the **Waterfall Software Development Methodology**, in which each phase is completed before moving to the next. The development process was carried out through a sequence of stages, including requirements analysis, system design, implementation, integration of AI components, testing, and documentation. This structured approach ensured that project requirements were clearly defined from the beginning, allowing the team to systematically develop and validate each component before proceeding to the next phase. The Waterfall model provided a well-organized development process that facilitated effective project management and thorough documentation throughout the project lifecycle.

1.6 Tools and Technologies Used

1.6.1 Software Technologies

- **React.js** – Frontend user interface.
- **FastAPI** – Backend REST API development.
- **PostgreSQL** – Relational database management.
- **Backblaze B2 Cloud Storage** – Cloud-based storage for datasets, uploaded files
- **SQLAlchemy** – Database ORM.
- **Pandas** – Data manipulation and preprocessing.
- **NumPy** – Numerical computations.
- **Scikit-learn** – Machine learning preprocessing algorithms.
- **DSPy & Large Language Models (LLMs)** – AI explanation engine and Natural Language Processing (NLP).
- **Python** – Backend implementation.
- **JavaScript** – Frontend implementation.
- **Git & GitHub** – Version control and collaboration.

1.6.2 Hardware Requirements

- Processor: Intel Core i3 or higher.

- RAM: Minimum 8 GB.
- Storage: At least 2 GB available space for the application, dependencies, and temporary datasets.
- Operating System: Windows 10/11 or Linux or macOS.
- Internet connection for accessing cloud-based Large Language Model (LLM) APIs and downloading project dependencies.

1.7 Project Timeline / Gantt Chart

- The project timeline is divided into multiple phases, each representing a major activity required for developing the proposed system. The details of these phases, including activities and estimated durations, are presented in Table 1-1.

Phase	Activities	Duration
Phase 1	Problem Identification	July - August 2025
Phase 2	Literature Review and Competitor Analysis	July - August 2025
Phase 3	Requirements Analysis	August- September 2025
Phase 4	Technology Research and Learning	October 2025 – March 2026
Phase 5	AI & Machine Learning Development	September 2025 – February 2026
Phase 6	System Design (UI/UX & Architecture)	November 2025 – February 2026
Phase 7	Backend Development	January – April 2026
Phase 8	Database Integration & API Development	February – April 2026
Phase 9	Frontend Development	March – May 2026
Phase 10	System Integration & Human in the Loop Integration	April – May 2026
Phase 11	Testing & Validation	May – June 2026
Phase 12	Documentation & Final Presentation	June 2026

Table 1-1 Project Timeline and Development Phases

A visual representation of the project schedule and the relationship between different phases over time is illustrated in Figure 1-1.

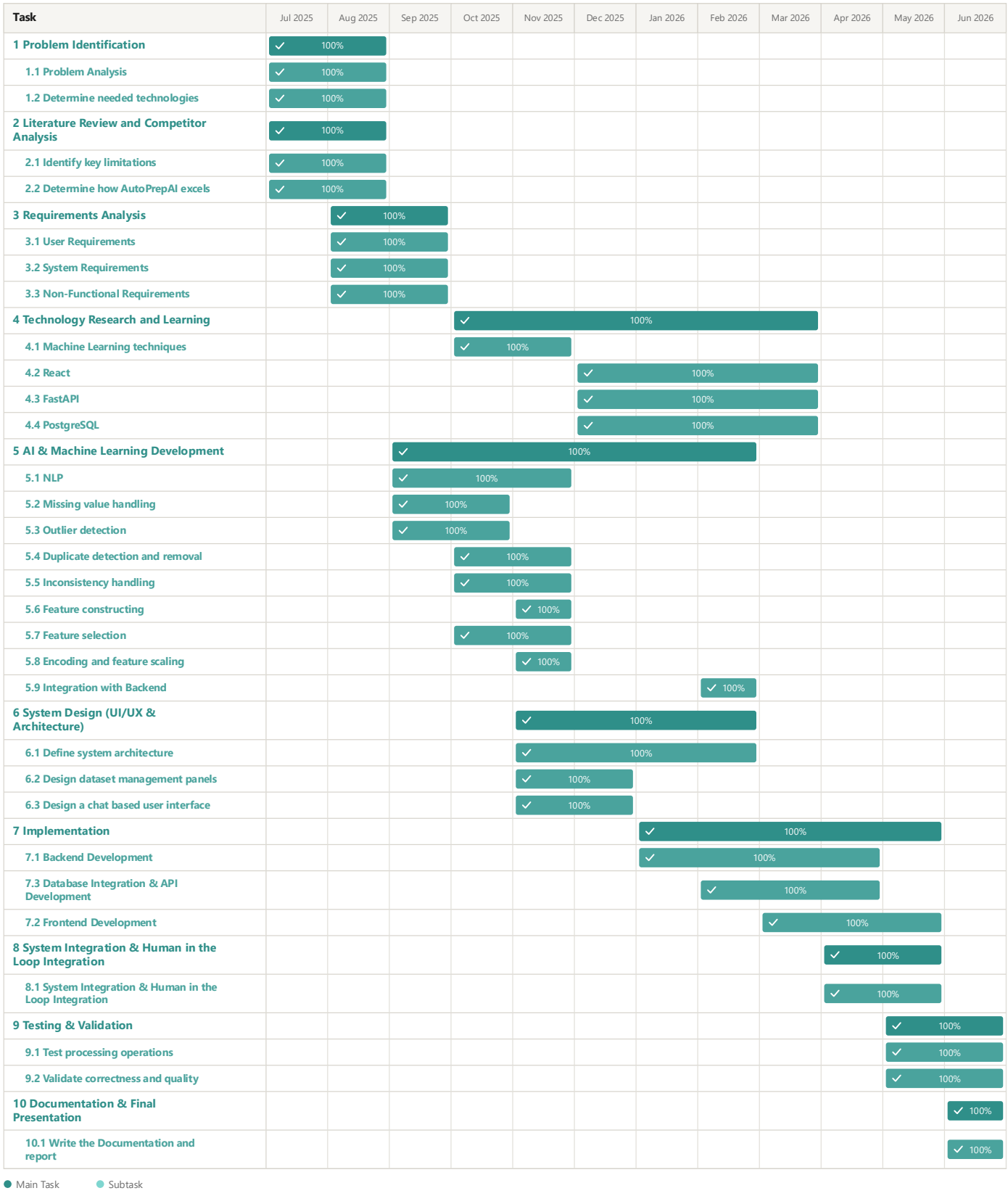


Figure 1-1 Project Timeline Gantt Chart

1.8 Report Organization

This report is organized into seven chapters. **Chapter One** introduces the project by presenting the background, problem definition, objectives, scope, development methodology, tools and technologies, project timeline, and report organization. **Chapter Two** reviews related work by discussing existing research and systems in the field of AI-assisted data preprocessing, followed by a comparative analysis highlighting the unique contributions of AutoPrepAI. **Chapter Three** presents the system analysis, including stakeholder analysis, functional and non-functional requirements, use case diagrams, and the feasibility study. **Chapter Four** describes the system design, including the overall architecture, class and sequence diagrams, database design, user interface mockups, and security considerations. **Chapter Five** explains the implementation of the system, covering the development environment, key implementation details, and sample outputs from the developed application. **Chapter Six** presents the testing and evaluation process, including the testing strategy, test cases, performance evaluation, and identified limitations. Finally, **Chapter Seven** concludes the report by summarizing the project's achievements, discussing the lessons learned throughout the development process, and presenting recommendations for future enhancements.

Chapter 2. Related Work

2.1 Existing Solutions

Data preprocessing is a crucial stage in the machine learning pipeline, as the quality of data directly affects the performance and reliability of predictive models. Various commercial platforms and intelligent assistants have been developed to facilitate data preparation; however, most existing solutions focus on either model optimization, rule-based transformations, or general AI assistance.

2.1.1 DataRobot and H2O Driverless AI

DataRobot [6] and H2O [7] Driverless AI are enterprise-grade AutoML platforms that automate model training and optimization. Although these systems perform preprocessing internally, their primary objective is to maximize model performance rather than provide transparent and user-oriented data cleaning. Consequently, preprocessing operations are mostly hidden from users and offer limited explainability.

2.1.2 Trifacta and Alteryx

Trifacta [8] and Alteryx [9] are widely adopted data wrangling and ETL tools. They provide visual interfaces for constructing transformation pipelines and defining cleaning rules. Despite their flexibility, these systems require considerable user intervention and rely mainly on rule-based operations, making them more suitable for experienced data engineers than casual users or researchers.

2.1.3 Numerous.ai

Numerous.ai [10] is an AI-powered assistant designed for spreadsheet environments such as Google Sheets and Microsoft Excel. It enables users to manipulate and transform data through natural language prompts. However, its capabilities are primarily focused on cell-level productivity rather than comprehensive dataset-level cleaning and analysis.

2.1.4 General-Purpose AI Assistants

Large language models and AI assistants, such as ChatGPT, GitHub Copilot, and Gemini, have recently demonstrated remarkable capabilities in code generation and analytical support. Nevertheless, these systems are intended for general-purpose applications and are not specifically designed for data preprocessing tasks. Their recommendations are often generic

and depend heavily on user prompts, lacking dedicated mechanisms for semantic duplicate detection, inconsistency resolution, and explainable cleaning workflows.

2.2 Comparative Analysis

Table 2-1 summarizes the main differences between existing solutions and AutoPrepAI.

Feature / Aspect	DataRobot / H2O	Trifacta / Alteryx	General AI Assistants	AutoPrepAI
Primary Focus	AutoML and Model Optimization	Data Wrangling and ETL	General AI Assistance	Intelligent Data Cleaning
Pipeline Orientation	Model-centric	Transformation-centric	Context-centric	Data-centric
Level of Automation	High	Medium	Prompt-based	High
Explainability	Limited	Limited	Generic Responses	Human-Readable Explanations
Semantic Duplicate Detection	Not Supported	Exact Matching Only	Not Specialized	Supported
Inconsistency Resolution	Limited	Rule-based	Generic	Context-aware
Agentic Architecture	Not Supported	Not Supported	Not Supported	Supported
Target Users	Enterprise ML Teams	Data Engineers	General Users	Researchers and Developers
Cost and Accessibility	High	Commercial	Medium	Academic-Friendly
Research Suitability	Medium	Low	Medium	High

Table 2-1 Comparison between existing solutions and AutoPrepAI

2.3 Research Gap and Motivation

Although numerous tools support different aspects of data preparation, most current solutions either focus on model optimization, depend heavily on manual rule definition, or provide only generic AI assistance. In addition, existing systems often address individual preprocessing problems independently and lack a unified and explainable framework for handling noisy and inconsistent datasets.

General-purpose AI assistants provide broad contextual knowledge but are not specialized for data cleaning and preprocessing tasks. Their responses are highly dependent on user prompts and do not incorporate dedicated mechanisms for identifying semantic duplicates, filtering noisy records, or resolving inconsistencies.

Therefore, there is a need for an intelligent and explainable framework that integrates multiple preprocessing functionalities while reducing manual effort and improving data quality.

2.4 Key Innovations of AutoPrepAI

AutoPrepAI introduces several features that distinguish it from existing solutions:

- **Agentic AI Architecture:** The system consists of specialized agents that cooperate to perform different preprocessing tasks automatically.
- **Semantic Duplicate Detection:** Unlike traditional exact matching methods, AutoPrepAI employs embedding-based techniques to detect records that represent the same entity despite textual variations.
- **Hybrid Noise Filtering:** Statistical methods and machine learning approaches are combined to identify noisy records and outliers more effectively.
- **Explainable AI:** AutoPrepAI provides human-readable explanations for its preprocessing decisions, allowing users to understand why specific cleaning, transformation, or selection operations were performed, thereby improving transparency and trust.
- **Automated Feature Engineering:** The system automatically generates meaningful features through various transformation techniques, such as encoding categorical variables, extracting temporal features, creating interaction features, and performing data transformations to enhance model performance.
- **Context-Aware Inconsistency Resolution:** A graph clustering and LLM-based approach is used to detect and correct inconsistent categorical values.
- **Integrated Data Cleaning Framework:** Multiple preprocessing functionalities are combined within a single intelligent system, reducing the need for separate tools and manual intervention.

Chapter 3. System Analysis

3.1 Stakeholder Analysis

Stakeholders are individuals and organizations directly or indirectly affected by AutoPrepAI's development, deployment, and use. The project encompasses diverse stakeholder groups with distinct interests and responsibilities.

3.1.1 Primary Stakeholders

End Users

End users are the primary beneficiaries of the AutoPrepAI system. They comprise:

- **Data Scientists & Analysts**: Professionals who spend significant time on data cleaning and preprocessing, requiring efficient tools to accelerate data preparation workflows and reduce manual effort.
- **Students & Researchers**: Academic users learning data preprocessing concepts or conducting research requiring clean datasets. They benefit from the system's educational value and detailed explanations of preprocessing operations.
- **Non-Technical Users**: Business analysts, domain experts, and operational staff who lack extensive programming expertise but need to prepare datasets for analysis. AutoPrepAI's intuitive interface reduces the barrier to entry for this group.
- **Machine Learning Engineers**: Professionals deploying ML models who require preprocessed, high-quality datasets as input.

Development Team

The project team consists of:

- **Lead Developer**: Responsible for overall system architecture, backend implementation (FastAPI), database design, and integration of preprocessing agents.
- **Frontend Developer**: Implements the React user interface, chat-based interaction, and visualization components.
- **ML Engineer / Data Scientist**: Designs and implements the six preprocessing agents and their underlying algorithms.
- **QA & Testing**: Ensures system reliability, performance, and correctness of

preprocessing operations across diverse datasets.

3.1.2 Secondary Stakeholders

Academic Supervisors & Evaluators

Faculty members and academic evaluators guide project development, assess technical quality, ensure adherence to academic standards, and validate research contributions. They have authority to approve or reject the system based on academic criteria.

Organizations & Educational Institutions

- **Corporations**: Companies relying on data-driven decision-making can adopt AutoPrepAI to reduce data preparation overhead, improve data quality, and accelerate analytics projects.
- **Educational Institutions**: Universities can deploy AutoPrepAI as a learning platform to teach data preprocessing concepts, data quality issues, and automated ML pipelines to students.
- **Research Organizations**: Entities conducting large-scale data analysis benefit from automated, reproducible preprocessing workflows.

Technology & Service Providers

- **Groq API Provider**: Supplies the LLM service for intent detection and explanations (Chat Mode).
- **Backblaze B2**: Provides cloud storage for dataset uploads and preprocessing artifacts.
- **PostgreSQL / Database**: Supports persistent session management and user authentication.
- **SMTP Email Service**: Handles user account verification and password reset notifications.

3.1.3 Tertiary Stakeholders

Open Source Community

Contributors and open source enthusiasts can extend AutoPrepAI with additional agents, preprocessing strategies, or UI enhancements, benefiting the broader data science community.

3.2 Functional Requirements

Functional requirements specify the capabilities and operations AutoPrepAI must provide to fulfill stakeholder needs.

3.2.1 User-Level Requirements

AutoPrepAI shall allow users to:

1. **Dataset Upload & Preview**
    - Upload tabular data in CSV or Excel (.xlsx) formats
    - Preview datasets before preprocessing to understand structure, dimensions, and sample records
    - Validate file format and structure before processing begins
2. **Mode Selection**
    - Choose from three interaction modes: Chat Mode, Auto Mode, or Manual Mode
    - Switch modes or restart the pipeline without losing session state
3. **Chat-Based Interaction**
    - Describe preprocessing tasks in natural language (e.g., "Remove outliers from salary column")
    - Have the system parse intent using DSPy and Groq LLM to automatically trigger appropriate agents
    - Receive clarification requests when your input is unclear or not supported.
4. **Check-Box Mode Control**
    - Select preprocessing operations independently using a modular agent interface
    - Configure agent parameters (e.g., outlier detection method, imputation strategy, feature selection threshold)
5. **Auto Mode Execution**
    - Trigger end-to-end preprocessing without manual intervention

- Have the system automatically execute all seven agents in a logical sequence
- Receive a complete action history upon completion

6. **Data Quality Issue Handling** AutoPrepAI shall address common data quality problems:
    - **Missing Values**: Detect and handle gaps using multiple imputation strategies (mean, median, KNN, MICE, categorical mode)
    - **Duplicate Records**: Detect exact and semantically similar duplicates and remove them
    - **Outliers**: Identify anomalies using configurable detection strategies (IQR, Z-score, Isolation Forest)
    - **Type Inconsistencies**: Correct data type mismatches and inconsistent categorical values
    - **Spelling Errors:** Correct misspelled text values using NLP-based spell correction and context-aware language models while preserving domain-specific terminology when applicable
    - **Scaling & Encoding**: Apply feature scaling (StandardScaler, MinMaxScaler) and categorical encoding (One-Hot, Label Encoding)
7. **Action Review & Approval**
    - View before/after comparisons for each preprocessing step
    - Receive AI-generated explanations justifying each recommended action
    - Accept or reject individual actions before final dataset is produced
    - Undo rejected changes and revisit previous preprocessing steps
8. **Operation History & Logging**
    - Access a detailed history of all applied preprocessing operations
    - View timestamps, affected features, and operation parameters for each action
9. **Authentication & Session Management**
    - Register a user account with email verification

- Log in securely using JWT-based authentication
- Reset password via email confirmation
- Maintain persistent session state across browser sessions
- Access previous datasets and preprocessing histories

10. **Data Download & Export**

- Download the cleaned dataset in the original format (CSV or Excel)
- Rename output files before downloading

3.2.2 System-Level Requirements

The AutoPrepAI system shall:

1. **Data Ingestion & Validation**

- Accept CSV and Excel file uploads up to a configurable size limit
- Validate file format and structure, rejecting malformed inputs
- Detect and report data schema issues (missing headers, inconsistent row lengths)
- Parse diverse data types: numerical, categorical, and mixed types

2. **Dataset Analysis**

- Automatically analyze uploaded datasets to detect characteristics:
  - Data types and distributions for each column
  - Missing value counts and percentages
  - Duplicate record counts (exact and semantic)
  - Outlier detection and anomaly scores

3. **Intent Detection (Chat Mode)**

- Use DSPy and Groq LLM to parse natural language commands
- Map user intents to preprocessing agents (e.g., "remove outliers" → Outlier Filter agent)
- Handle multi-step intents by decomposing into agent workflows

- Generate a clarification message when the system cannot interpret the user's request

4. **Preprocessing Agent Execution** Implement ten specialized preprocessing agents:

a) Data Standardizer Agent

- Detect and correct data type inconsistencies within columns
- Normalize categorical values (spelling corrections, case normalization)
- Handle mixed-type columns by converting to consistent formats
- Apply configurable standardization rules

b) Data Type Inconsistency Agent

- Detect mixed or conflicting data types within individual columns using parallel multi-strategy analysis
- Implement type detection strategies for common data types:
    - Numeric detection (integers, floats, scientific notation)
    - Datetime detection (ISO 8601, US/EU date formats, timestamp formats)
    - Boolean detection (true/false, yes/no, 1/0 representations)
    - String detection (text, categorical, alphanumeric)
- Identify inconsistencies by counting detected type frequencies per column
- Recommend a target type using rule-based thresholds over the detected type of distribution in each column.
- Test conversion feasibility before applying type transformations:
    - Validate numeric conversion success rate
    - Verify datetime parsing compatibility
    - Flag values that cannot be safely converted
- Estimate confidence based on the expected conversion failure rate (High, Medium, or Low confidence).

- Convert inconsistent columns to the recommended data type while preserving existing missing values and treating unconvertible values as missing.
- Execute detection in parallel for multiple columns to optimize performance.
- Record detected types, recommended types, conversion confidence, conversion failures, and representative examples of values that could not be standardized to ensure transparent preprocessing decisions.

c) Duplicate Remover Agent

- Detect exact row duplicates
- Identify semantically similar records using similarity metrics (cosine)
- Retain one representative record and remove others

d) Outlier Filter Agent

- Implement multiple outlier detection strategies:
    - Interquartile Range (IQR) method
    - Z-score method
    - Isolation Forest algorithm
- Remove detected outliers based on user choice

e) Missing Value Handler Agent

- Detect missing values (NaN, None, empty strings)
- Implement multiple imputation strategies:
    - Simple imputation (mean, median, mode)
    - K-Nearest Neighbors (KNN) imputation
    - Multivariate Imputation by Chained Equations (MICE)
- Choose strategy based on data type and missingness pattern
- Document imputation decisions for transparency

f) Feature Engineer Agent

- Suggest meaningful derived features using LLM guidance

- Log feature engineering operations with justifications

i) Feature Selector Agent

- Remove irrelevant or redundant features based on:
    - Statistical independence tests
    - Feature importance scores (Random Forest, mutual information)
- Support configurable feature selection thresholds

j) Scaling Agent

- Apply numerical feature scaling techniques:
    - StandardScaler (z-score normalization)
    - MinMaxScaler (normalization to [0,1])
    - RobustScaler (robust to outliers)
- Support configurable scaling methods for selected columns

k) Encoding Agent

- Encode categorical variables using:
    - One-Hot Encoding for nominal features
    - Label Encoding for ordinal features

5. **Explanation & Justification**

- Provide clear, human-readable explanations for each preprocessing recommendation
- Use LLM-generated summaries to justify agent decisions
- Explain the impact of each operation on data quality

6. **State Management & Persistence**

- Maintain session state across multiple HTTP requests (stateful pipeline)
- Store intermediate DataFrames in cloud storage (Backblaze B2)
- Persist user sessions in PostgreSQL with SQLAlchemy ORM

- Support multi-user isolation with session-based conversation IDs
- Enable pausing and resuming preprocessing workflows

7. **User Access Control**
    - Implement JWT-based authentication for secure API endpoints
    - Enforce role-based access control (optional: admin, user roles)
    - Prevent unauthorized access to other users' datasets and sessions
    - Support email verification for account security
8. **API Endpoints**
    - **Authentication**
        - **/auth/signup**: Register a new user account
        - **/auth/login**: Authenticate user and generate session/token
        - **/auth/verify-email**: Verify user email address
        - **/auth/resend-verification**: Resend email verification link
        - **/auth/forgot-password**: Request password reset
        - **/auth/reset-password**: Reset user password
        - **/auth/logout**: Terminate user session
    - **Chat Interface (Core Interaction Layer)**
        - **/chat**: Submit natural language preprocessing requests and receive responses from the system
        - **/chat/feedback**: Submit feedback on chat responses or generated preprocessing results
    - **Conversations Management**
        - **/conversations**: Retrieve all user conversations
        - **/conversations/{conversation_id}**: Retrieve a specific conversation
        - **/conversations/{conversation_id}**: Delete a conversation
        - **/conversations/{conversation_id}/rename**: Rename a conversation

- **File Processing**
    - **/download/{path}**: Download processed dataset file
- **System Health**
    - **/health**: Check system status

9. **Error Handling & Validation**
    - Validate all user inputs (file formats, parameter ranges, intent clarity)
    - Provide clear, actionable error messages for invalid operations
    - Gracefully handle edge cases (empty datasets, all-null columns)
    - Log errors for debugging and system monitoring

3.3 Non-Functional Requirements

Non-functional requirements define quality attributes and system constraints.

3.3.1 Usability

- **Intuitive Interface**: Provide a chat-based and manual control interface that requires minimal training
- **Clear Feedback**: Offer immediate visual and textual feedback for user actions
- **Accessibility**: Ensure the web interface is accessible to users with varying technical expertise
- **Responsive Design**: Ensure the React UI is functional on desktop

3.3.2 Performance

- **Response Time**: Preprocessing operations on datasets up to 50,000 rows should be completed in under 45 seconds for most operations
- **Throughput**: Support concurrent sessions for multiple users without significant degradation
- **Scalability**: Design the pipeline to handle datasets up to 60 MB with modular processing and efficient algorithms
- **API Latency**: API endpoints should respond within 5000 ms under normal load

- **Memory Efficiency**: Avoid loading entire datasets into memory; use streaming where possible for large files

3.3.3 Reliability

- **Consistency**: Preprocessing operations must produce reproducible results across multiple runs on identical data
- **Data Integrity**: Ensure no data loss during upload, processing, or download
- **Session Persistence**: Maintain session state reliably across network interruptions
- **Availability**: Target 99% uptime for the system during operational hours
- **Backup & Recovery**: Implement automated backups of user datasets and session states

3.3.4 Security & Privacy

- **Authentication**: Enforce JWT-based authentication for all protected endpoints
- **Data Encryption**: Transmit data over HTTPS/TLS and encrypt sensitive data at rest
- **Access Control**: Enforce multi-user isolation; users can only access their own datasets and sessions
- **Input Validation**: Validate all user inputs to prevent SQL injection, file type attacks, and malicious uploads
- **Password Security**: Hash passwords using strong algorithms (Argon2); enforce password reset policies
- **Audit Logging**: Log all user actions (login, upload, preprocessing, download) for compliance and auditing

3.3.5 Maintainability

- **Modular Architecture**: Organize code into distinct modules for agents, services, routes, and utilities
- **Code Documentation**: Provide comprehensive comments and docstrings for all major functions and classes
- **Version Control**: Use Git with clear commit messages and branching strategy

- **Technology Stack**: Leverage established frameworks (FastAPI, React, SQLAlchemy) for long-term support
- **Testing**: Implement unit tests, integration tests, and end-to-end tests for critical components
- **Extensibility**: Design the agent framework to allow adding new preprocessing agents with minimal changes to core pipeline

3.3.6 Scalability

- **Cloud Storage**: Leverage Backblaze B2 for unlimited scalable storage of datasets and artifacts

3.4 Use Case Diagrams & Actor Descriptions

3.4.1 Use Case Diagram Overview

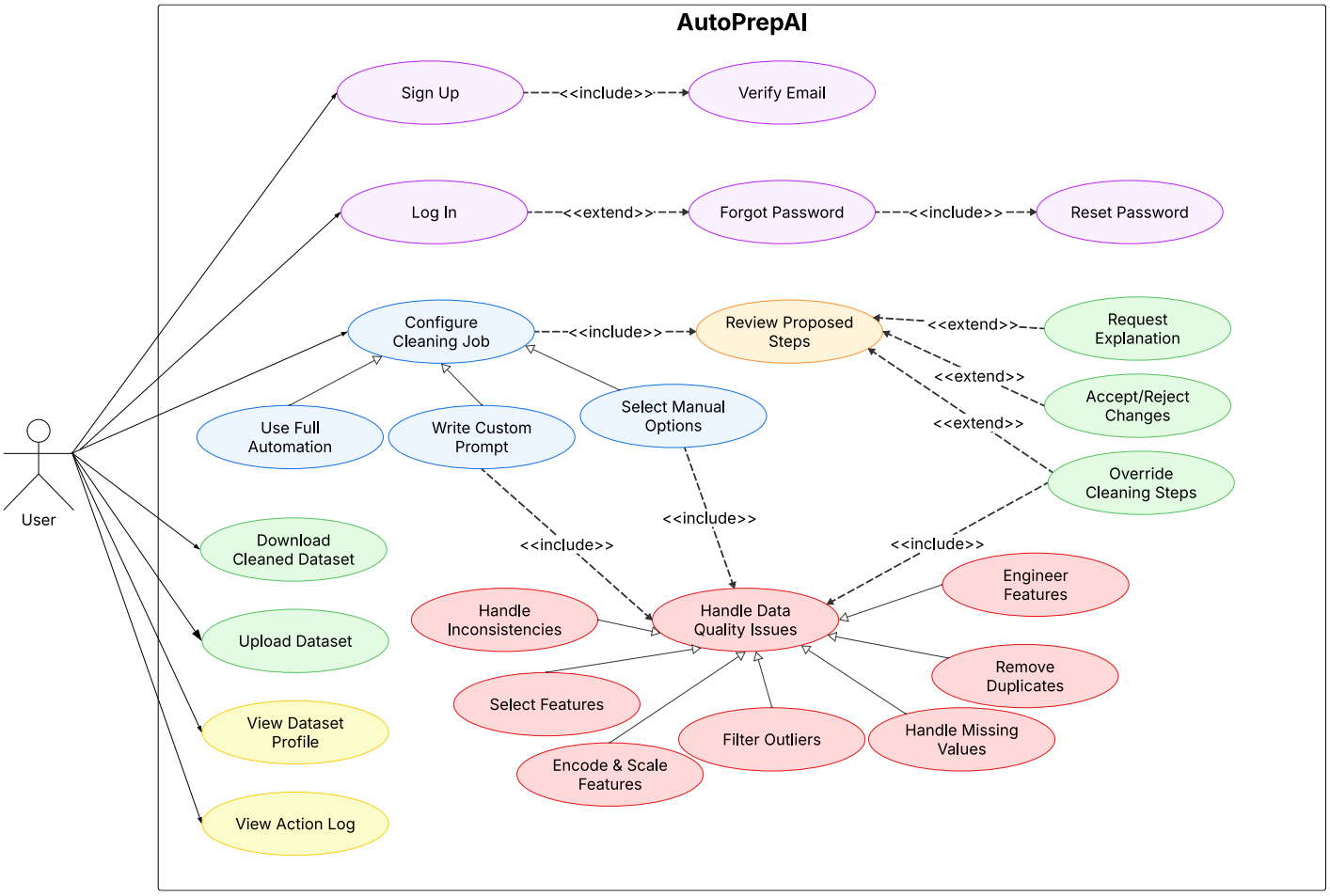


Figure 3-1 Use-case Diagram

3.4.2 Actor Descriptions

Actor 1: Data Scientist / Data Analyst

Profile: Technical professional with programming expertise and domain knowledge in data preparation.

Goals:

- Clean and preprocess datasets efficiently to reduce manual effort
- Apply domain-specific preprocessing strategies
- Understand and validate preprocessing decisions
- Export clean data for downstream analysis

Use Cases Engaged:

- Register & Login (account management)
- Upload Dataset (CSV/Excel with large volumes)
- Preview Dataset & analyze data quality issues
- Execute Manual Mode with custom agent configurations
- Review & Approve Actions with detailed scrutiny
- Download Cleaned Dataset & operation logs
- Export historical preprocessing logs for reproducibility

Actor 2: Non-Technical User / Business Analyst

Profile: Domain expert without programming background; needs data preparation but lacks technical skills.

Goals:

- Clean datasets without writing code
- Understand what preprocessing operations are being applied
- Gain confidence in data quality improvements
- Access cleaned data for business reporting or analysis

Use Cases Engaged:

- Register & Login (account management)
- Upload Dataset (small to medium datasets)
- Preview Dataset (visual understanding)
- Execute Auto Mode (minimal configuration needed)
- Review & Approve Actions (with clear, non-technical explanations)
- Download Cleaned Dataset (final export)

Interaction Preferences: Intuitive UI, plain-English explanations.

Actor 3: Student / Researcher

Profile: Academic user learning data preprocessing concepts or conducting research on small datasets.

Goals:

- Learn data preprocessing techniques and best practices
- Experience with different preprocessing strategies
- Understand why each preprocessing operation is necessary
- Evaluate the system's effectiveness on diverse datasets

Use Cases Engaged:

- Register & Login
- Upload Dataset (academic datasets)
- Preview Dataset & explore data quality issues
- Execute Chat Mode (exploratory, natural language instructions)
- Execute Manual Mode (try different configurations)
- Review & Approve Actions (understand justifications)
- View Operation History (study preprocessing workflows)
- Download Cleaned Dataset (for downstream modeling)

Interaction Preferences: Educational explanations, step-by-step guidance.

Actor 4: System Administrator

Profile: Technical staff managing AutoPrepAI deployment, user accounts, and system health.

Goals:

- Monitor system performance and availability
- Manage user accounts and access control
- Ensure data security and compliance
- Troubleshoot system issues

Use Cases Engaged:

- Monitor API performance and uptime
- Review audit logs for user actions
- Manage user accounts and permissions
- Configure system parameters (storage limits, API keys, LLM settings)
- Backup and restore user data

Interaction Preferences: system logs.

3.4.3 Core Use Cases

Use Case 1: Register & Login

Aspect	**Description**
Actors	Non-authenticated user
Precondition	User has a valid email address
Main Flow	1. User navigates to registration page 2. Enters email and password 3. System sends verification email 4. User verifies email 5. Account created; user logs in with JWT token
Alternative Flow	User already has account → logs in directly with email/password
Postcondition	User authenticated; session token obtained; can access personalized features

Table 3-1 Use Case Description – Register and Login

Use Case 2: Upload Dataset

Aspect	Description
Actors	Authenticated user
Precondition	User is logged in; has a CSV or Excel file ready
Main Flow	1. User clicks "Upload Dataset" 2. Selects file from local system 3. System validates file format and size 4. Creates new preprocessing session 5. Stores file in Backblaze B2 6. Returns session ID and initial statistics
Alternative Flow	File validation fails → user receives error message with guidance for fixing
Postcondition	Dataset uploaded; session created; ready for preprocessing mode selection

Table 3-2 Use Case Description – Upload Dataset

Use Case 3: Preview Dataset

Aspect	Description
Actors	Authenticated user with uploaded dataset
Precondition	Dataset has been uploaded successfully
Main Flow	1. User requests dataset preview 2. System retrieves dataset from B2 (first 1,000 rows) 3. Displays statistics: row count, column count, data types 4. Shows sample data in table format
Alternative Flow	Dataset is very large → show preview of first N rows only
Postcondition	User understands dataset structure and quality issues; ready to proceed with preprocessing

Table 3-3 Use Case Description – Preview Dataset

Use Case 4: Execute Chat Mode

Aspect	Description
Actors	Authenticated user with uploaded dataset
Precondition	Dataset uploaded; user in Chat Mode
Main Flow	1. User types natural language command (e.g., "Remove duplicates from email column") 2. System uses DSPy + Groq LLM to parse intent

Aspect	Description
	3. Maps intent to appropriate agents 4. Executes agents on dataset 5. Presents the selected preprocessing workflow and a preview of the resulting data for user validation before execution. 6. Displays AI-generated explanations
Alternative Flow	Intent ambiguous → system returns a guidance message containing the supported preprocessing operations when a user request cannot be mapped to a valid intent.
Postcondition	Preprocessing actions recommended; user can approve/reject before finalizing

Table 3-4 Use Case Description – Execute Chat Mode

Use Case 5: Execute Auto Mode

Aspect	Description
Actors	Authenticated user with uploaded dataset
Precondition	Dataset uploaded; user selects Auto Mode
Main Flow	1. System automatically executes all six agents in sequence 2. Each agent processes data and identifies issues 3. System generates Explanations for each agent 4. Compiles comprehensive action history 5. Presents all actions to user for batch review
Postcondition	All preprocessing agents executed; results ready for user approval

Table 3-5 Use Case Description – Execute Auto Mode

Use Case 6: Execute Manual Mode

Aspect	Description
Actors	Authenticated user with uploaded dataset
Precondition	Dataset uploaded; user selects Manual Mode
Main Flow	1. User selects one agent to execute (e.g., Outlier Filter) 2. Configures agent parameters (e.g., detection method) 3. System executes agents with specified configuration 4. Returns results with explanations 5. User reviews and approves/rejects 6. User can download results

Aspect	Description
Postcondition	Selected agent executed; user maintains full control over preprocessing sequence

Table 3-6 Use Case Description – Execute Manual Mode

Use Case 7: Review & Approve Actions

Aspect	Description
Actors	Authenticated user reviewing preprocessing results
Precondition	Preprocessing agents have generated recommendations
Main Flow	1. System displays before/after data comparison 2. Provides AI-generated justification for each action 3. User reviews individual actions 4. User approves or rejects each action 5. Approved actions are applied; rejected actions are discarded
Postcondition	User-approved dataset ready for download

Table 3-7 Use Case Description – Review and Approve Actions

Use Case 8: View Operation History

Aspect	Description
Actors	Authenticated user
Precondition	Preprocessing operations have been completed
Main Flow	1. User requests operation history 2. System retrieves all preprocessing actions from session 3. Displays timeline with: operation type, timestamp, affected columns, parameters 4. User can filter by operation type or date range 5. User can export history as log file
Postcondition	User understands complete preprocessing workflow; can audit changes

Table 3-8 Use Case Description – View Operation History

Use Case 9: Download Cleaned Dataset

Aspect	Description
Actors	Authenticated user with approved preprocessing results
Precondition	User has reviewed and approved preprocessing actions
Main Flow	1. User clicks "Download Cleaned Dataset" 2. System finalizes all approved changes 3. Generates cleaned dataset in original format (CSV/Excel) 4. Initiates browser download
Postcondition	Cleaned dataset downloaded to user's local system

Table 3-9 Use Case Description – Download Cleaned Dataset

3.5 Feasibility Study

3.5.1 Technical Feasibility

Assessment: **HIGHLY FEASIBLE**

Rationale:

1. **Established Technology Stack**: FastAPI, React, PostgreSQL, and scikit-learn are mature, well-documented frameworks with strong community support.
2. **Available Libraries & Tools**:
    - DSPy for LLM pipelines and intent detection
    - Groq API for scalable, cost-effective LLM inference
    - Scikit-learn, pandas, NumPy for preprocessing algorithms
    - SQLAlchemy for ORM simplifying database operations
    - Backblaze B2 for scalable cloud storage
3. **Preprocessing Algorithms**: All six preprocessing agents are implemented using well-established, proven algorithms:
    - Duplicate detection: hash-based and similarity-based methods
    - Outlier detection: IQR, Z-score, Isolation Forest
    - Missing value imputation: mean, median, KNN, MICE

- Feature engineering & selection: standard ML techniques

4. **Architecture Design**: Stateful pipeline design with session persistence, multi-user isolation, and modular agent framework is technically sound and implementable.
5. **Risk Factors**:
    - **LLM Dependency**: Reliance on Groq API for intent detection introduces potential latency and cost variability → Mitigation: implement fallback intent detection or cached responses
    - **Data Volume Handling:** Large datasets (>50000×100 or >100×50000) are rejected entirely. Smaller datasets are loaded fully into memory with no chunked processing. → Mitigation: pre-upload size gate rejects oversized data before processing.
    - **Performance Scaling**: Multi-user concurrent access requires connection pooling and load balancing → Mitigation: use FastAPI async capabilities and database connection pooling

Conclusion: Technical feasibility is strong with manageable risks.

3.5.2 Economic Feasibility

The economic feasibility assessment evaluates the estimated costs required to develop and operate the proposed system. The analysis considers development costs, infrastructure expenses, and third-party service costs to determine the sustainability of the proposed cost model. The estimated costs are presented in Table 3-10.

Component	Cost Type	Estimated Cost
Development	One-time	0 (academic project)
Infrastructure	Recurring (monthly)	
Backblaze B2 Storage	~$0.006 per GB/month	~$60-150/month (10-25 TB)
PostgreSQL Hosting	Database service (AWS/Heroku)	~$50-200/month
Groq API	LLM inference calls	~$0.10-50/month (based on usage)
Email Service (SMTP)	Email delivery	Free/low-cost ($0-10/month)
Estimated Total Monthly Cost		**$110-410/month**

Table 3-10 Economic Feasibility Cost Analysis

Revenue Model Options (for future commercialization):

1. **Freemium**: Free tier for small datasets; premium tier for large datasets + advanced features → Estimated revenue: $1-5K/month per 100 active users
2. **Subscription**: Tiered subscriptions (Basic: $9.99, Professional: $29.99, Enterprise: Custom) → Estimated revenue: $2-10K/month per 100 active users
3. **Enterprise Licensing**: Customized solutions for organizations → Estimated revenue: $500-5K per organization

Profitability Timeline: With modest subscription adoption (50-100 paying users), the system could achieve profitability within 6-12 months.

Conclusion: Economic feasibility is strong; operational costs are reasonable and scalable revenue models exist.

3.5.3 Operational Feasibility

Assessment: **HIGHLY FEASIBLE**

Operational Requirements:

1. **Deployment Infrastructure**:
    - Containerized deployment (Docker) enabling easy scaling across cloud platforms (AWS, Google Cloud, Azure)
    - CI/CD pipeline (GitHub Actions) for automated testing and deployment
    - Load balancing and auto-scaling to handle variable user load
2. **Monitoring & Maintenance**:
    - Application Performance Monitoring (APM) tools to track API latency, error rates, and system health
    - Database query optimization and regular backups
    - Log aggregation and alerting for system errors and security events
    - Estimated maintenance effort: 5-10 hours/week for ongoing operations
3. **User Support**:
    - Help documentation, FAQs, and in-app tutorials
    - Email-based user support (estimated 2-5 hours/week initially, scaling with user

base)

- Community forum or Slack channel for user collaboration

4. **Data Governance & Compliance**:
    - GDPR(General Data Protection Regulation) compliance: user data deletion, privacy policies, data processing agreements
    - Data backup & disaster recovery procedures
    - Regular security audits and vulnerability assessments.
    - Audit logging for regulatory compliance
5. **Staffing Requirements**:
    - **Phase 1 (Launch)**: 1-2 developers for maintenance, 1 support person
    - **Phase 2 (Growth)**: Add DevOps engineer, QA, and additional support staff as user base grows

Risk Factors:

- **LLM Service Availability**: Groq API outages could disable Chat Mode → Mitigation: implement graceful degradation or fallback intent detection
- **Storage Scaling**: Rapid growth could exceed initial storage estimates → Mitigation: implement data archival policies and tiered storage
- **User Support Burden**: High user support load could overwhelm small team → Mitigation: invest in comprehensive documentation and knowledge base

Conclusion: Operational feasibility is high with manageable staffing and infrastructure requirements.

3.6 Summary of Feasibility Assessment

The feasibility assessment evaluates the proposed system across technical, economic, and operational dimensions to determine its practicality and potential for successful implementation. The summarized results are presented in Table 3-11.

Dimension	**Feasibility**	**Confidence**	**Key Risks**
Technical	High	90%	LLM dependency,

Dimension	Feasibility	Confidence	Key Risks
			data volume handling
Economic	High	85%	Revenue model validation, market adoption
Operational	High	90%	Support scaling, service availability
Overall	**HIGHLY FEASIBLE**	**90%**	Manageable and addressable

Table 3-11 Feasibility Assessment Summary of the Proposed System

3.7 Conclusion

The AutoPrepAI system is technically sound, economically viable, and operationally sustainable. The project successfully bridges the gap between automated ML preprocessing pipelines and user transparency through its Human-in-the-Loop architecture. With diverse stakeholder groups (data scientists, non-technical users, students) and a clear value proposition (automating tedious preprocessing while maintaining user control), AutoPrepAI is positioned for successful development and deployment.

The modular agent framework, cloud-native architecture, and scalable cost model support long-term growth and commercialization potential. The identified risks are manageable through targeted mitigation strategies, positioning the project as a viable contribution to both academic research and the broader data science community.

Chapter 4. System Design

4.1 System Architecture / Component Diagram

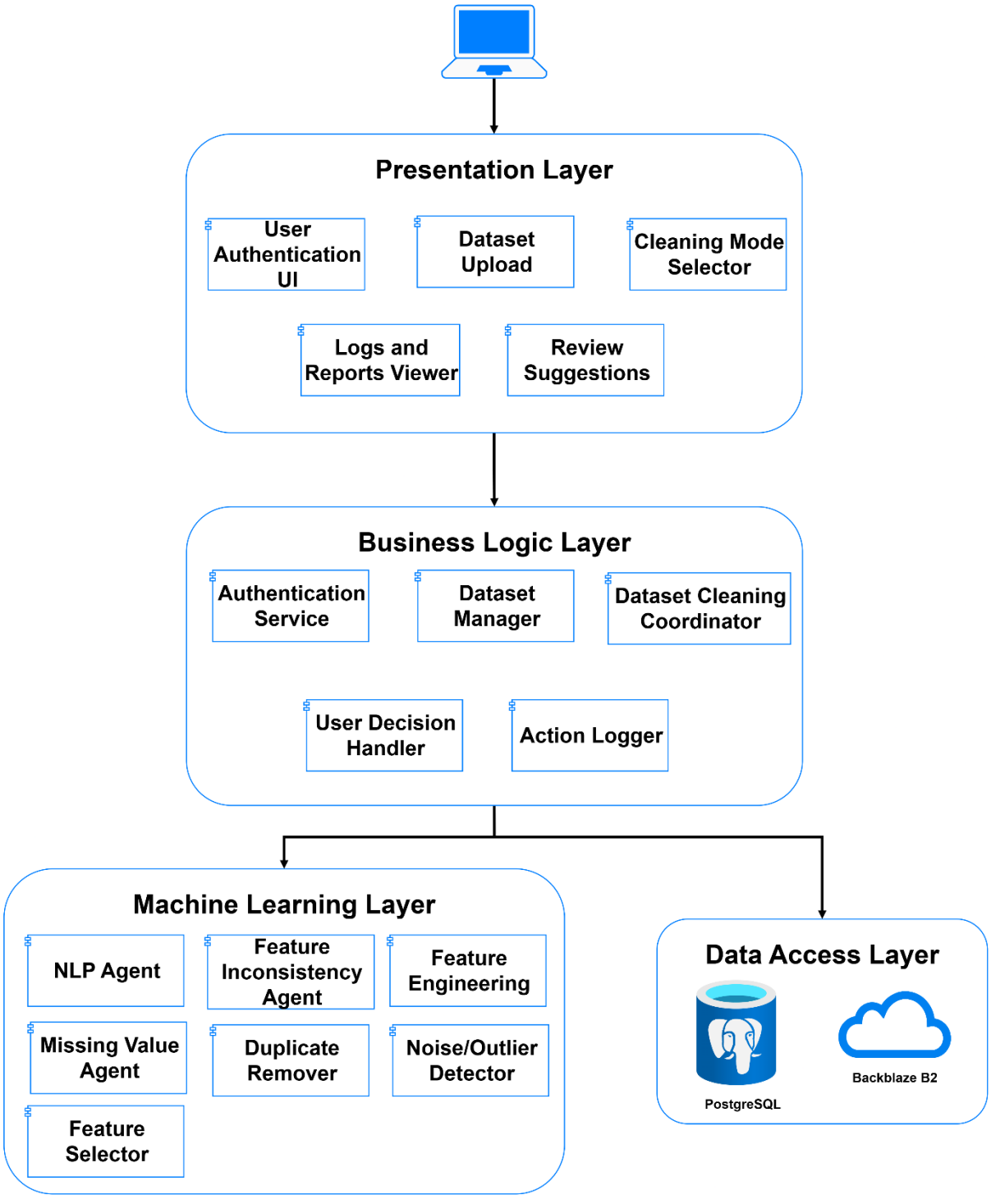


Figure 4-1 System Architecture

4.2 Class Diagrams

Pipeline Orchestration and Intent Processing Subsystem

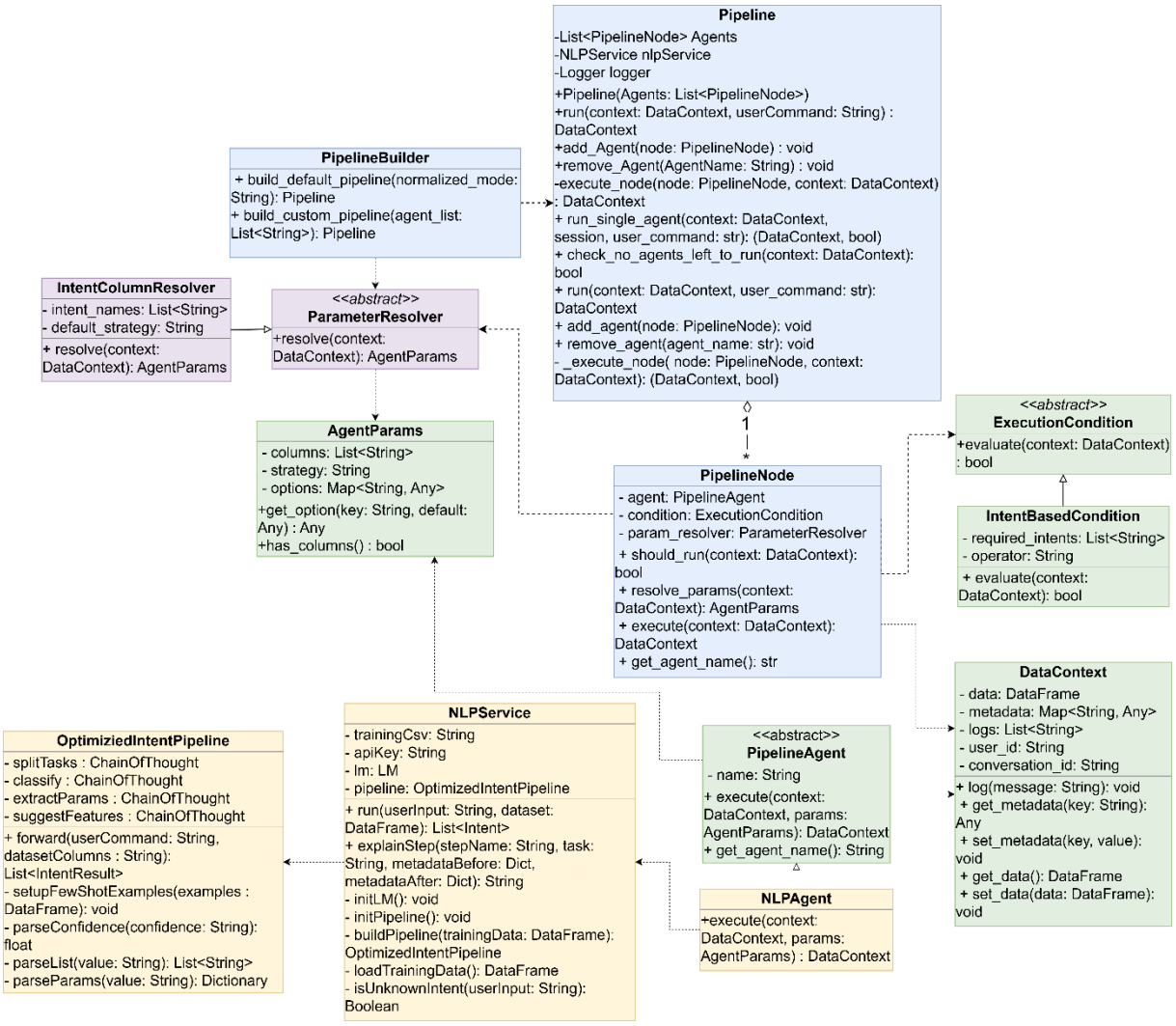


Figure 4-2 Pipeline Orchestration and Intent Processing Class Diagram

Pipeline Hierarchy Class Diagram

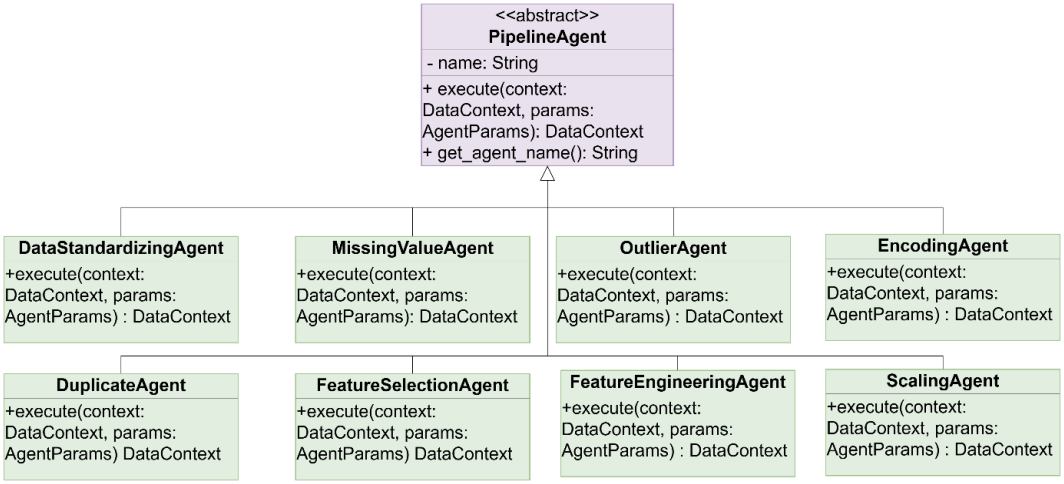


Figure 4-3 Pipeline Hierarchy Class Diagram

Missing Value and Outlier Filtering Diagram

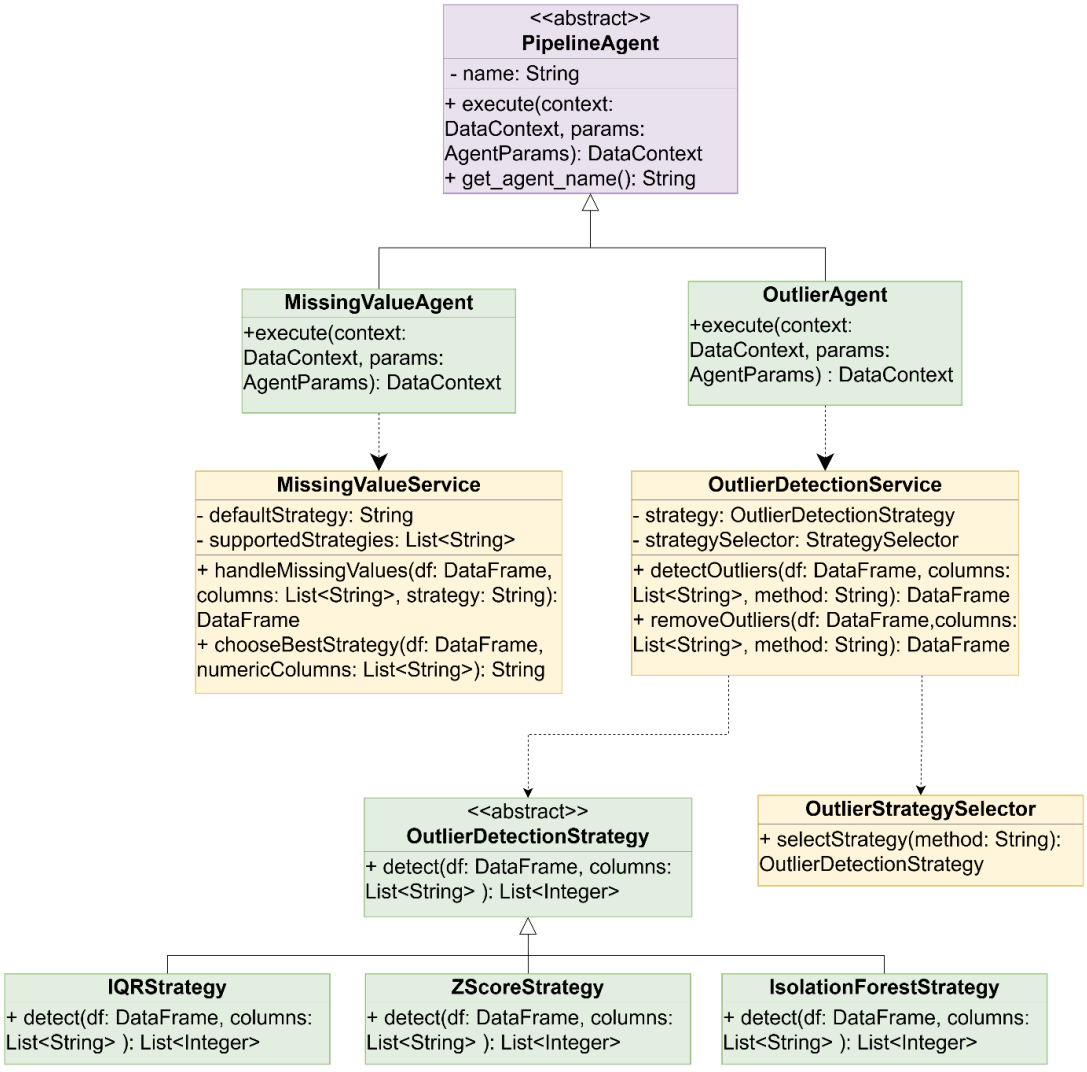


Figure 4-4 Missing Value and Outlier Filtering Class Diagram

Data Type Inconsistency Class Diagram

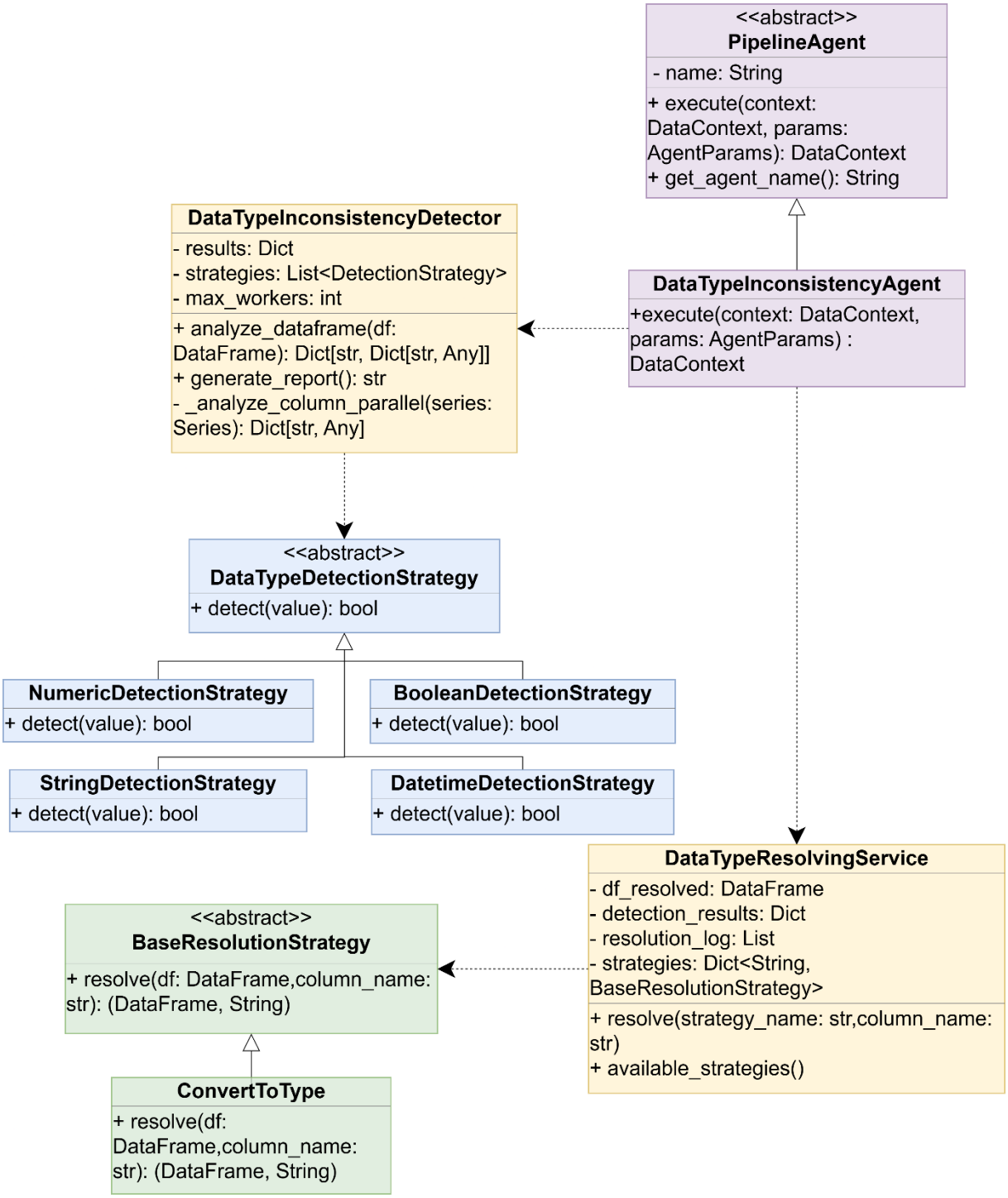


Figure 4-5 Data Type Inconsistency Class Diagram

Data Standardization Class Diagram

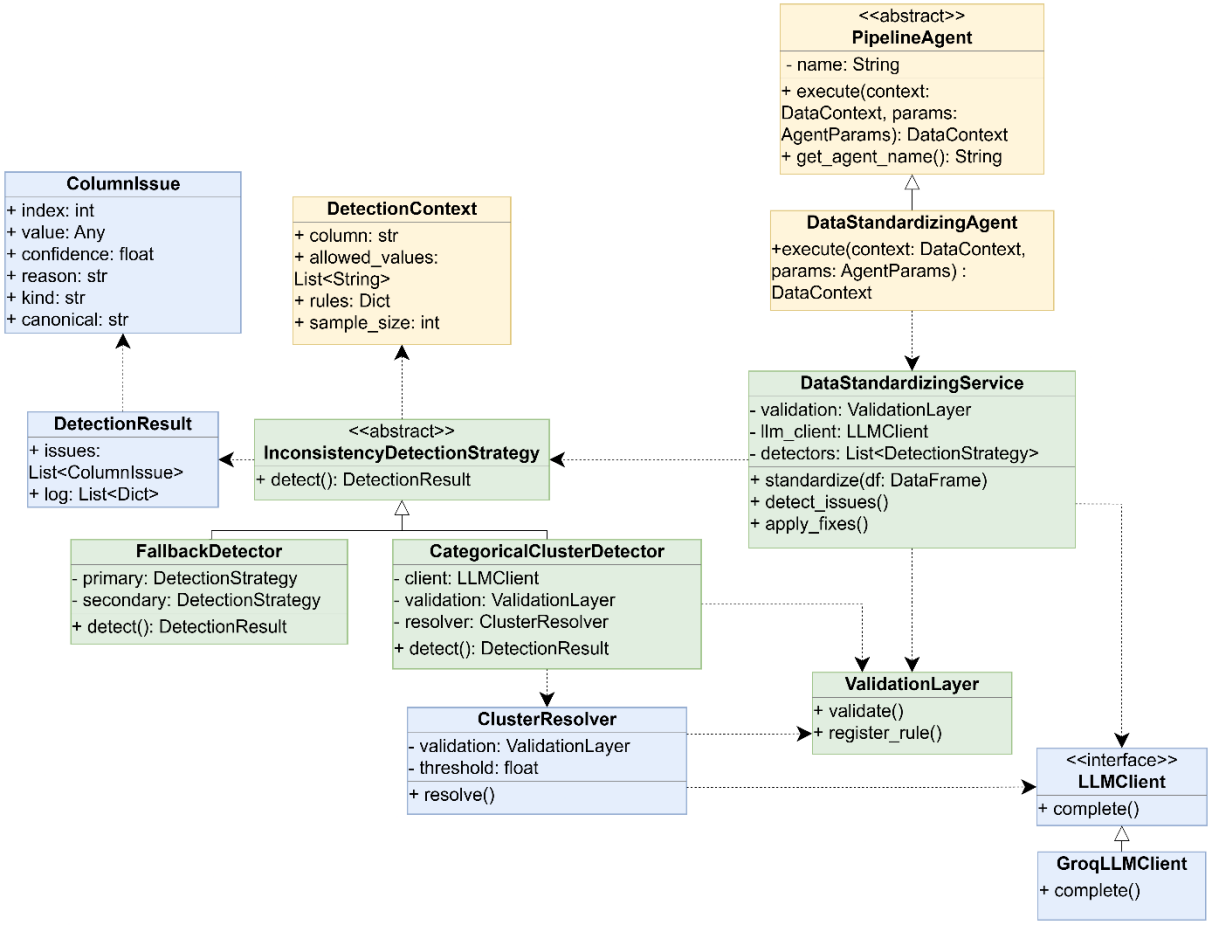


Figure 4-6 Data Standardization Class Diagram

Duplicate Removal Class Diagram

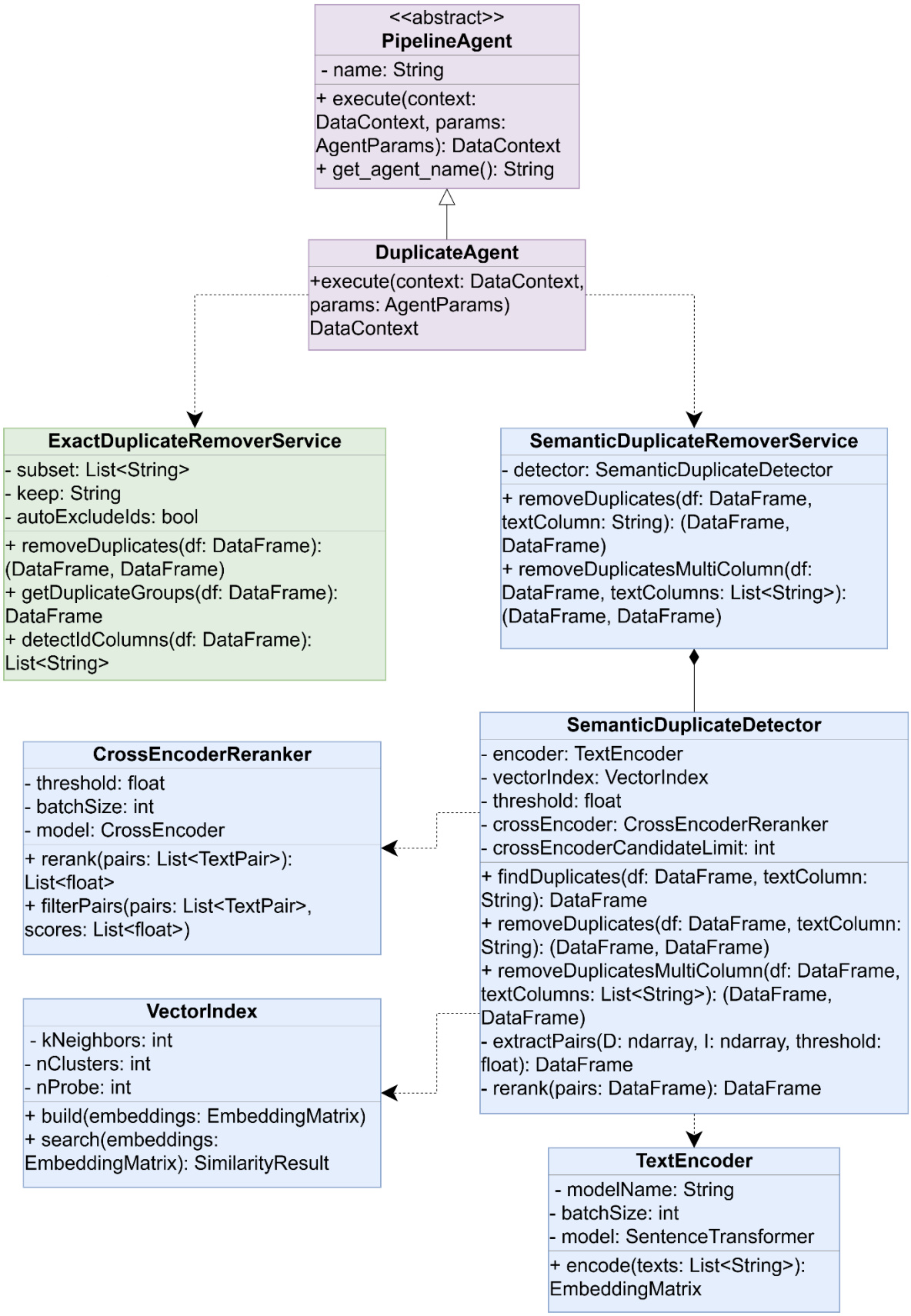


Figure 4-7 Duplicate Removal Class Diagram

Feature Selection and Feature Engineering Class Diagram

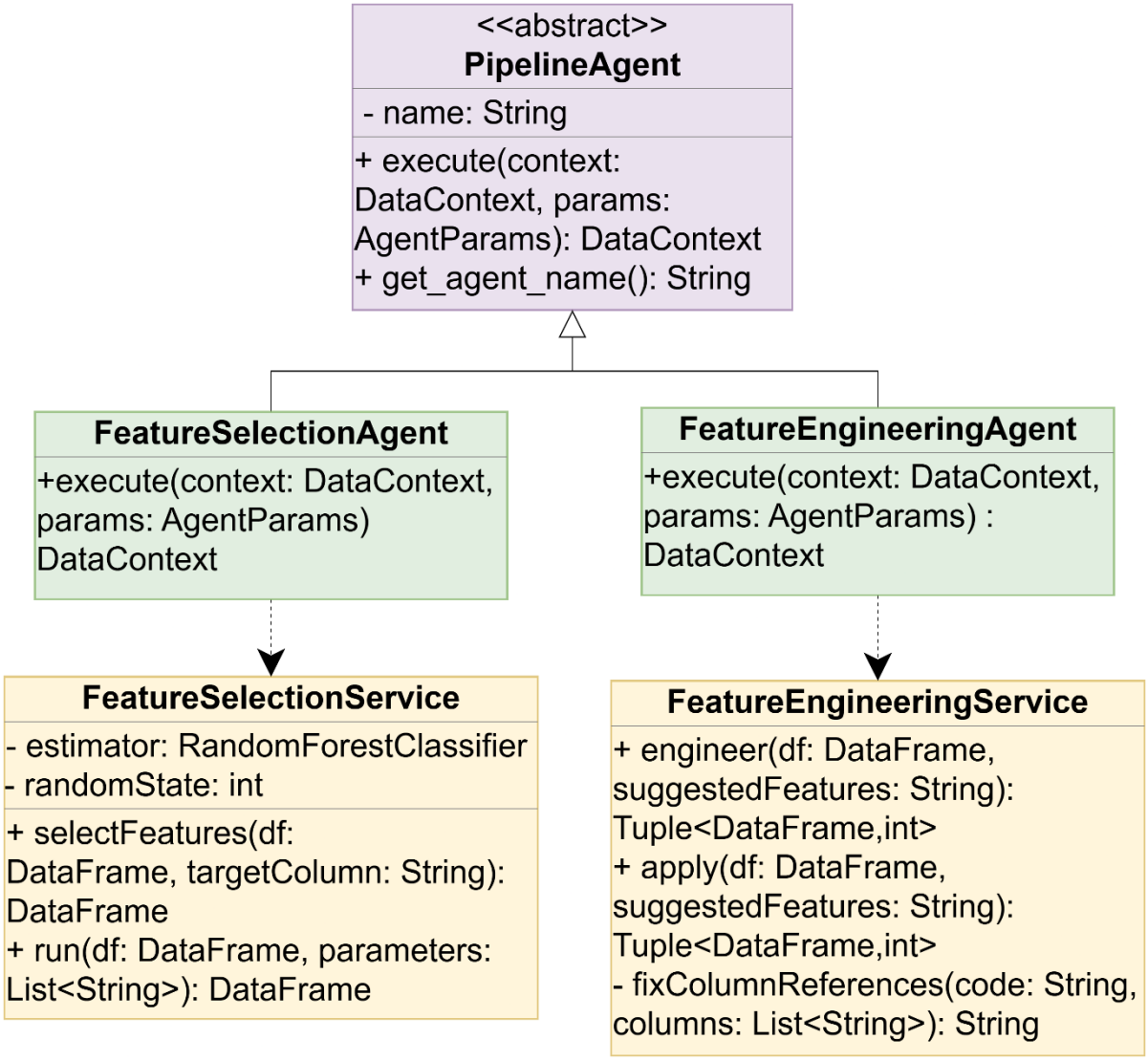


Figure 4-8 Feature Selection and Feature Engineering Class Diagram

Encoding and Scaling Class Diagram

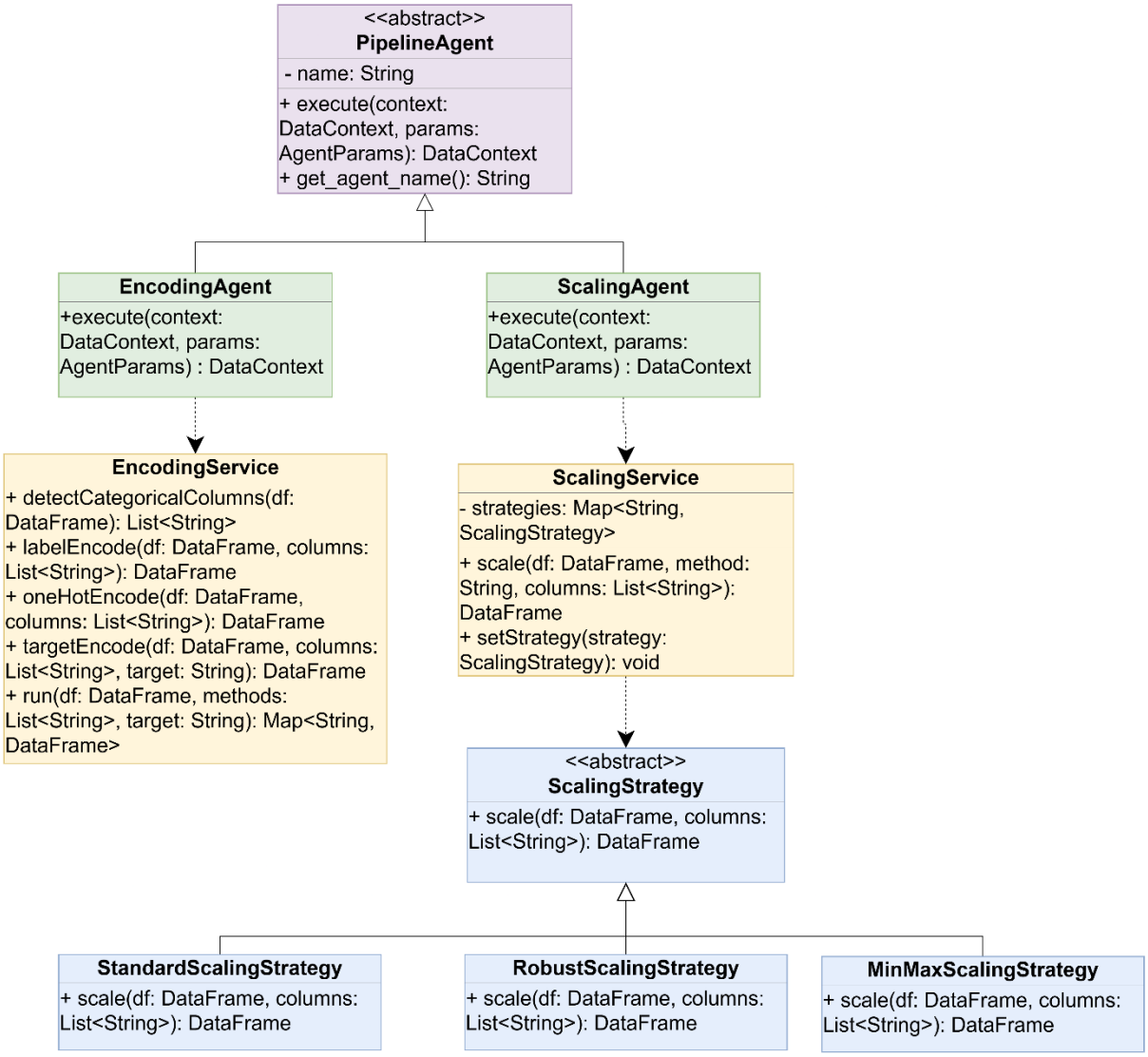


Figure 4-9 Encoding and Scaling Class Diagram

User Management and Authentication Class Diagram

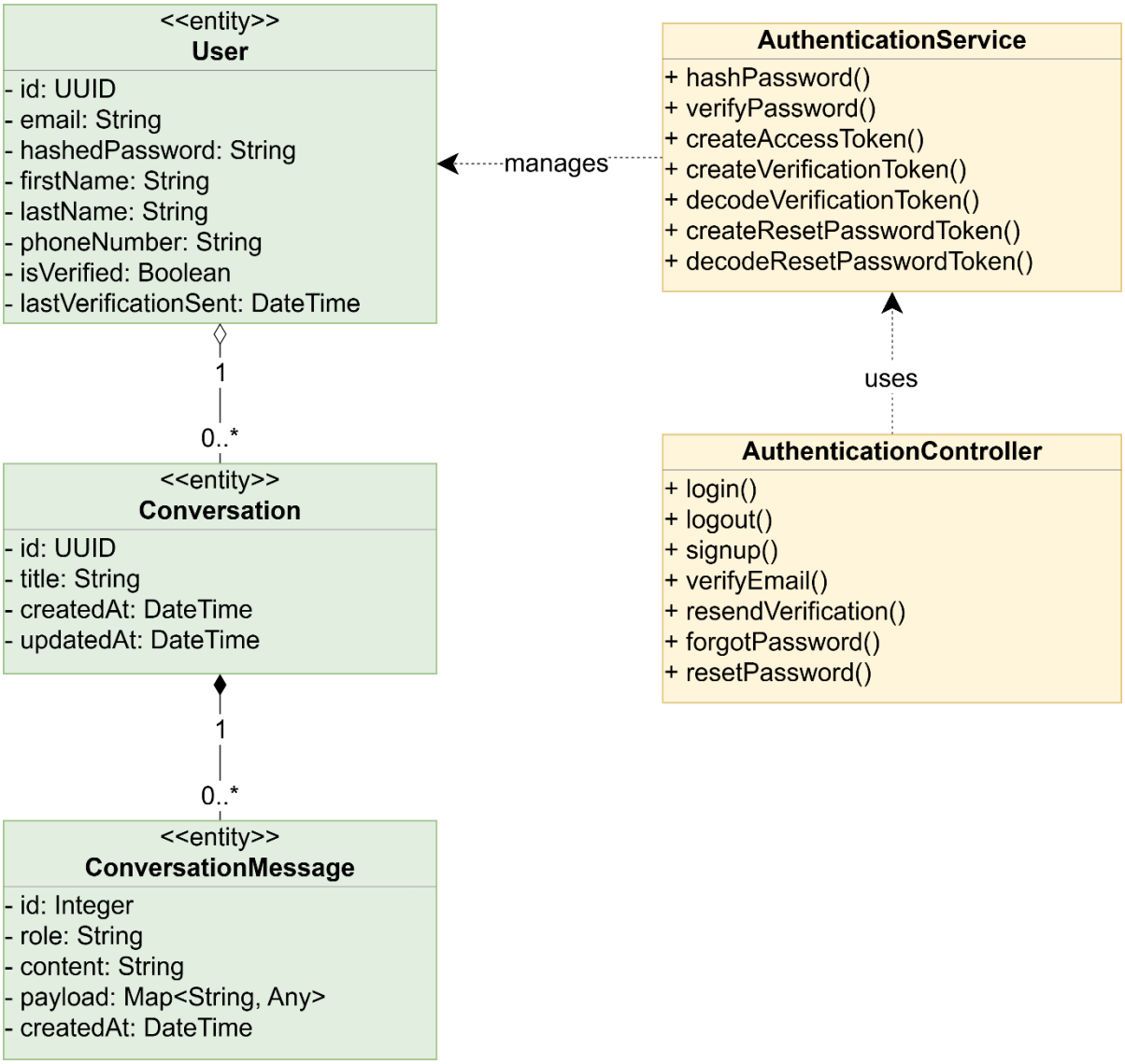


Figure 4-10 User Management and Authentication Class Diagram

4.3 Sequence Diagrams (for key use cases)

Prompt-Based Sequence Diagram

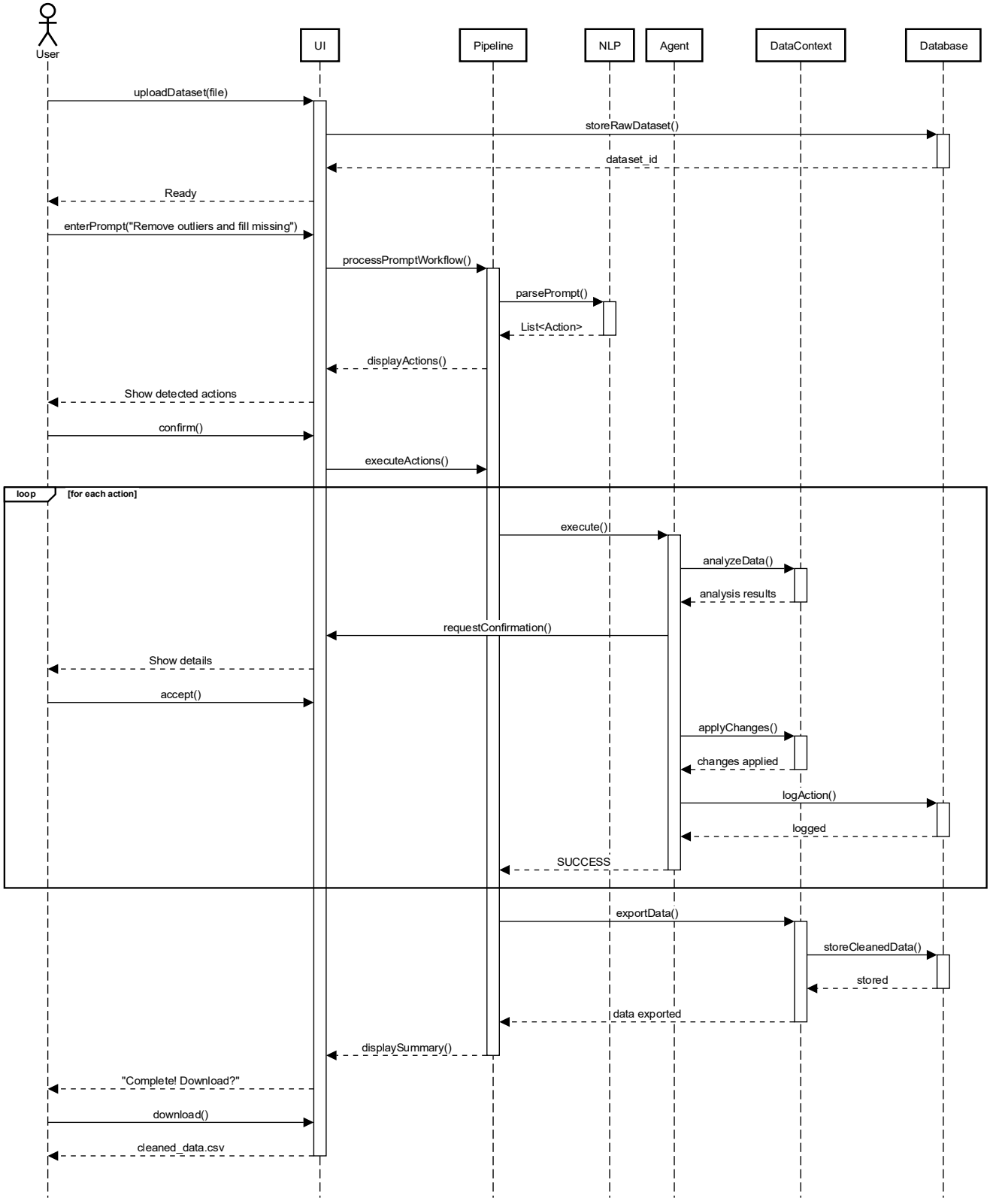


Figure 4-11 Prompt-based Sequence Diagram

Full Automation Sequence Diagram

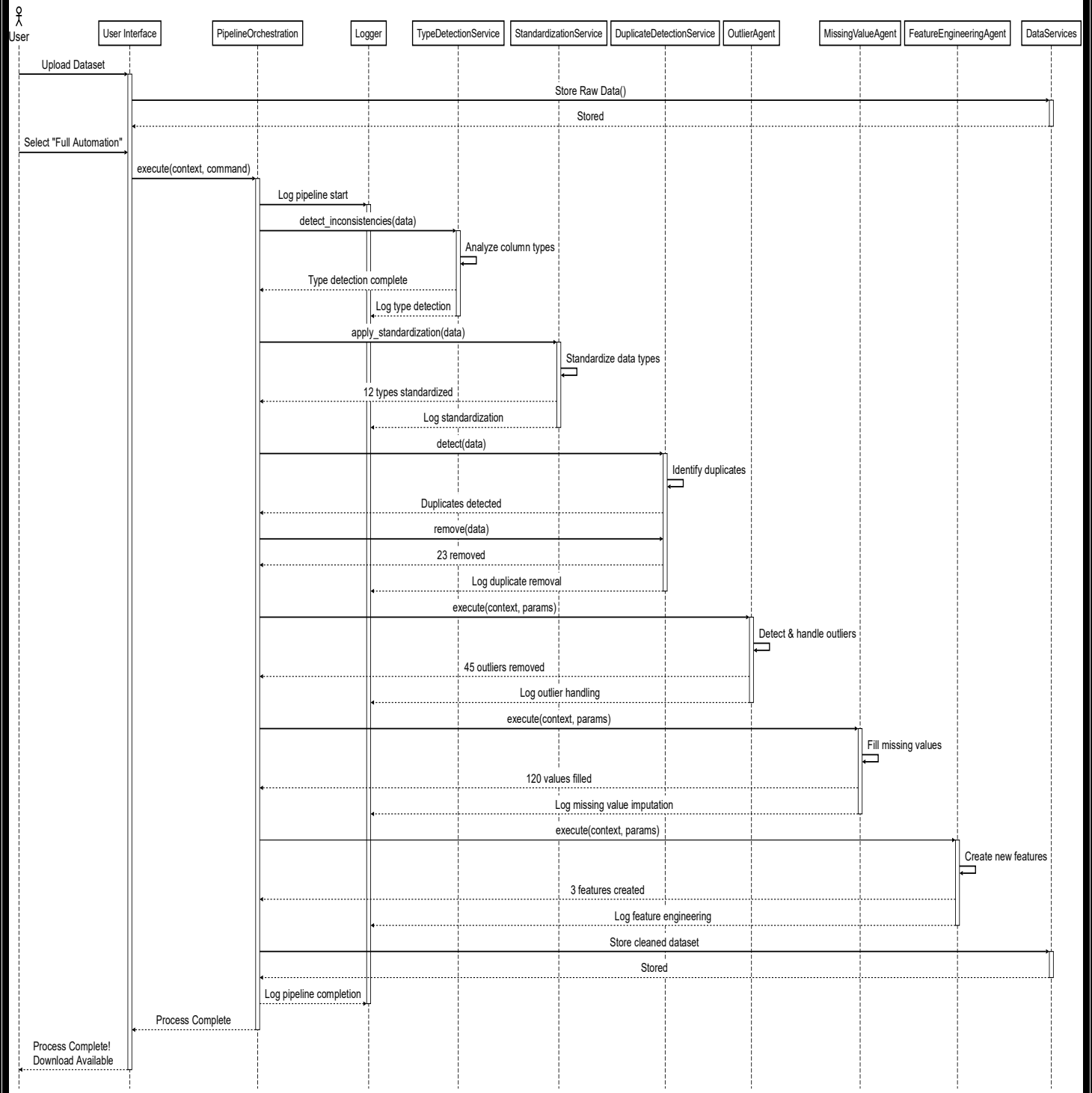
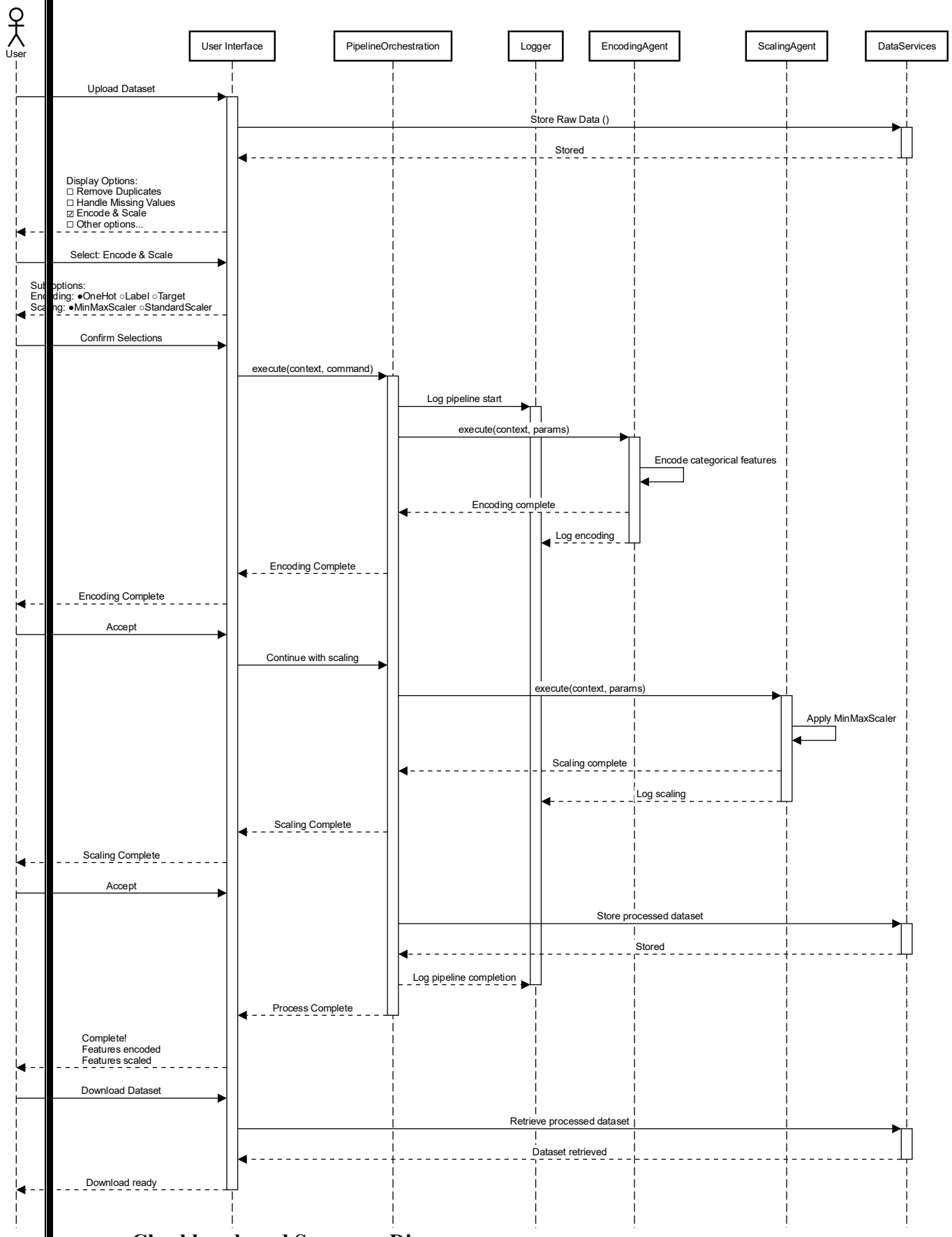


Figure 4-12 Full Automation Sequence Diagram



Checkbox-based Sequence Diagram

Figure 4-13 Checkbox-based Sequence Diagram

Duplicate Removal Sequence Diagram

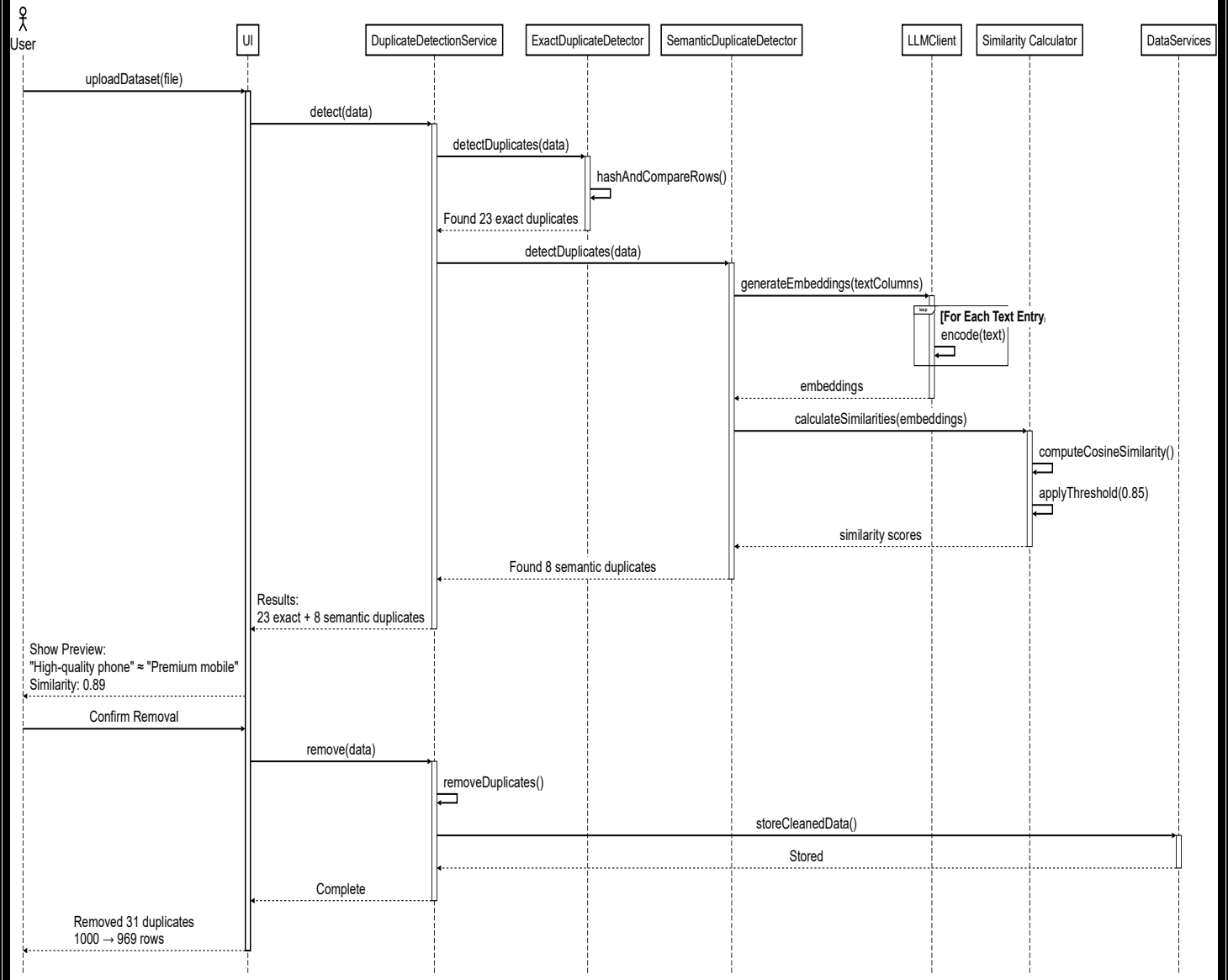


Figure 4-14 Duplicate Removal Sequence Diagram

Data Standardization Sequence Diagram

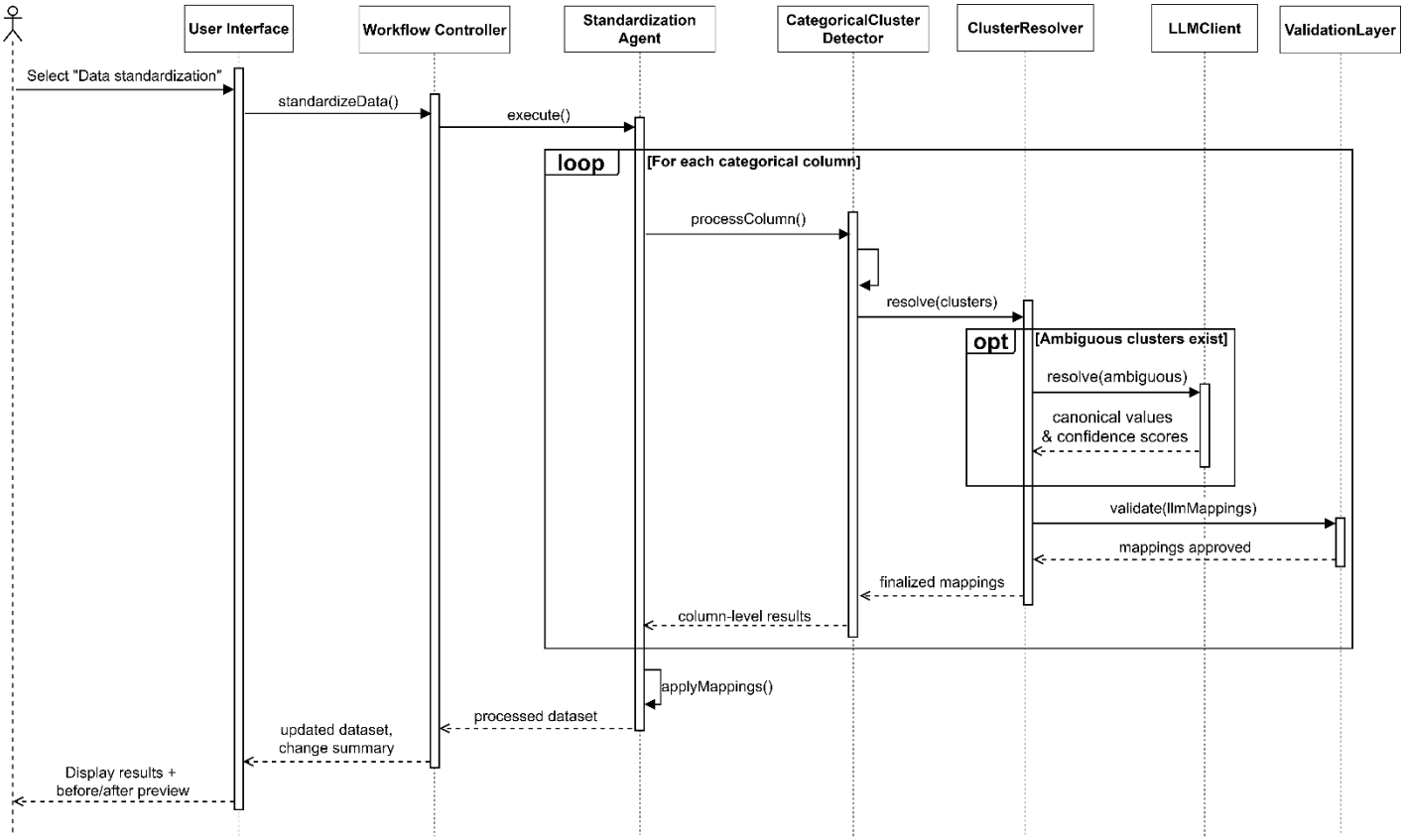


Figure 4-15 Data Standardization Sequence Diagram

Human in the Loop Sequence Diagram

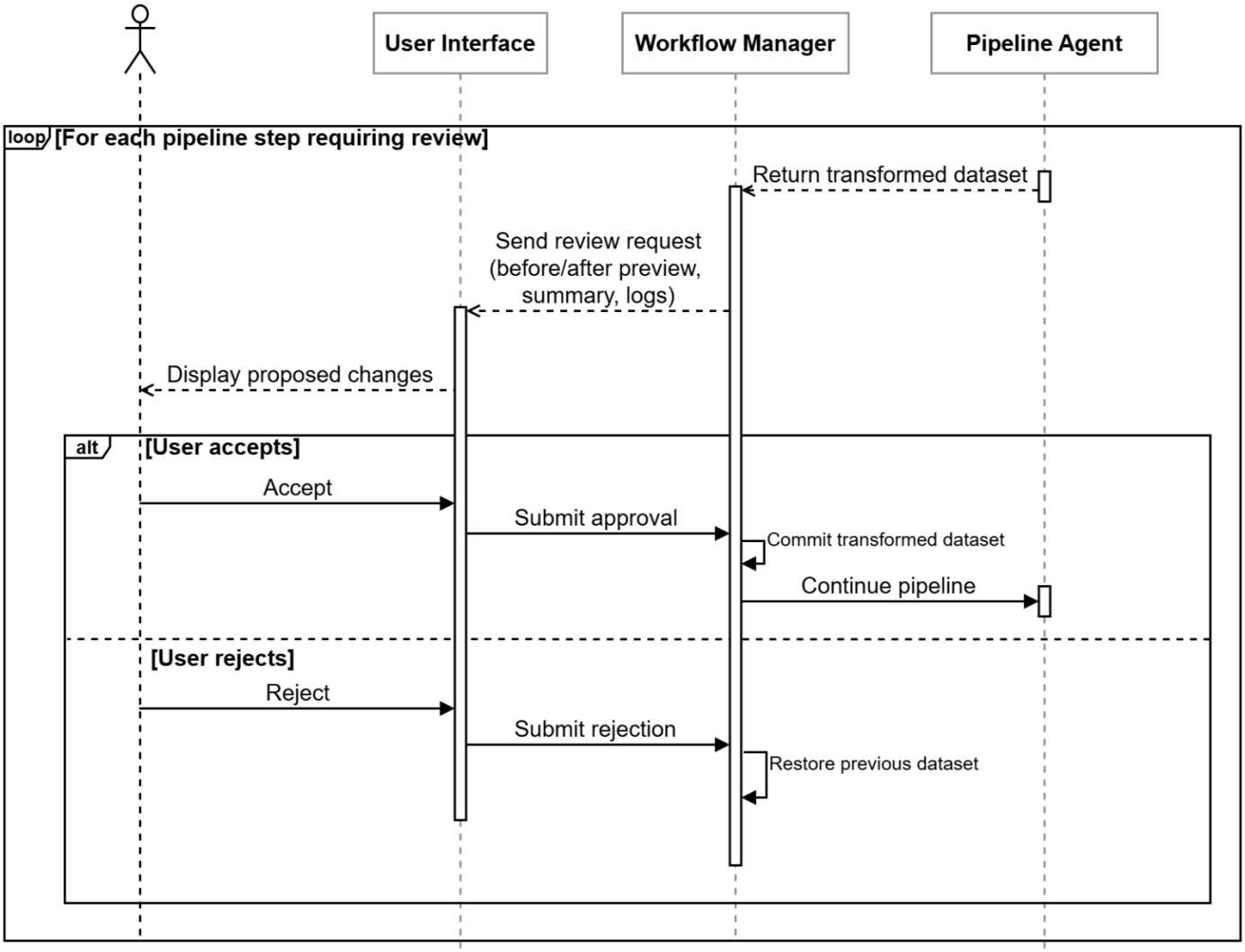


Figure 4-16 Human in the Loop Sequence Diagram

4.4 Database Entity-Relationship Diagram (ERD)

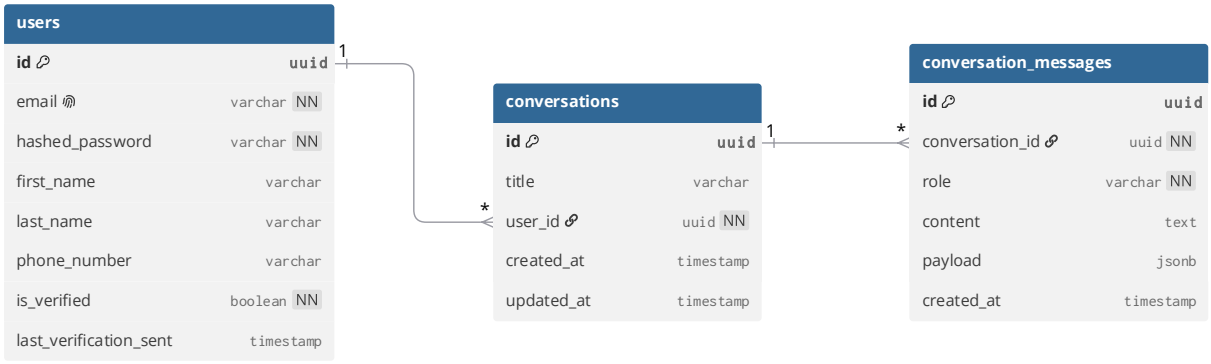


Figure 4-17 Database Entity-Relationship Diagram (ERD)

4.5 User Interface Wireframes / Mockups

Sign Up Page Mockup

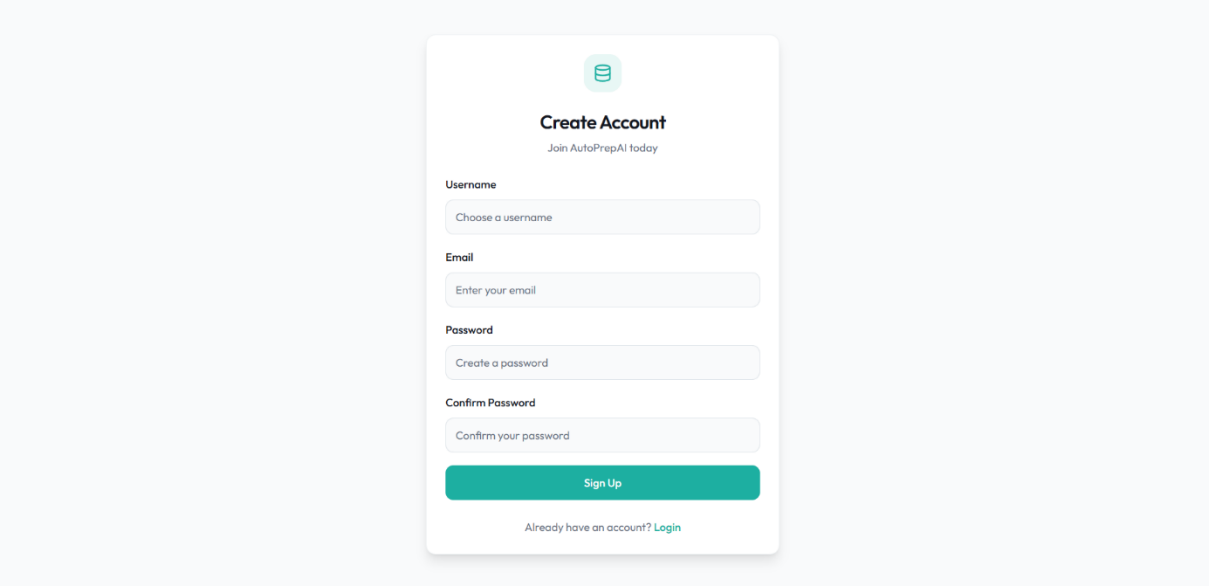


Figure 4-18 Signup Page Mockup

Login Page Mockup

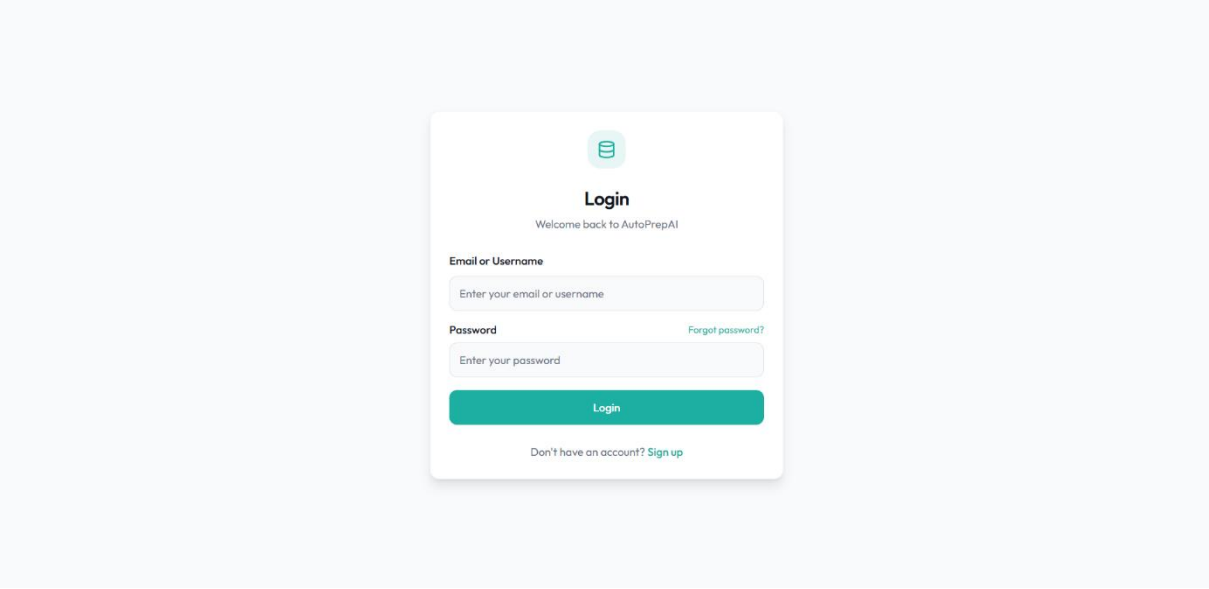


Figure 4-19 Login Page Mockup

Reset Password Mockup

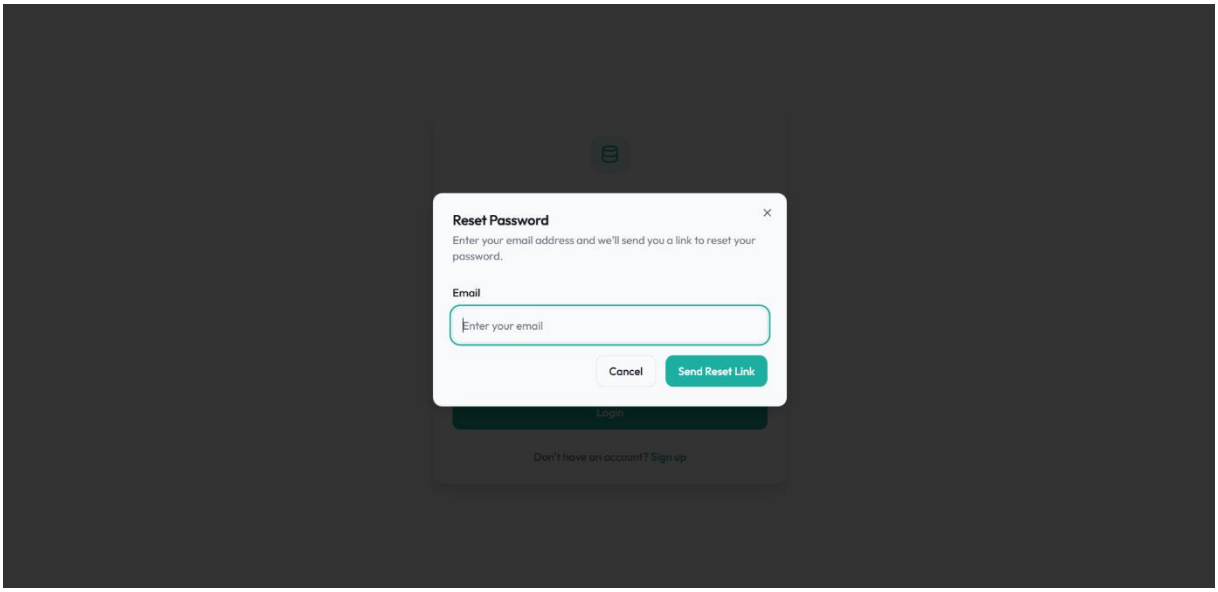


Figure 4-20 Reset Password Mockup

AutoPrepAI Main Dashboard Mockup

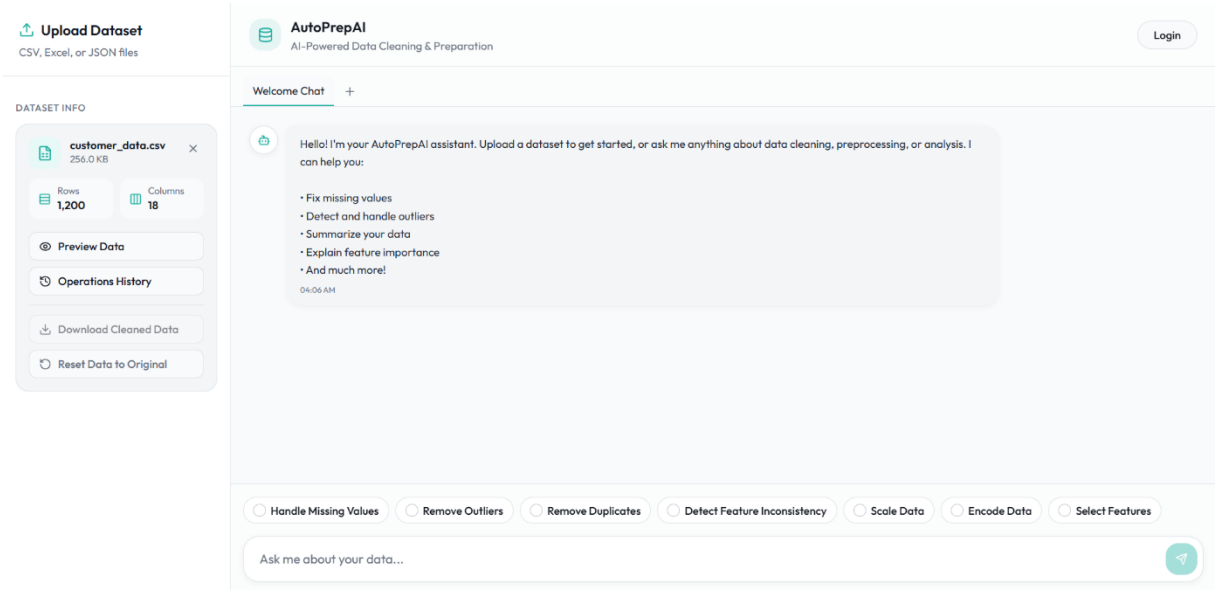


Figure 4-21 Main Dashboard Mockup

Dataset Preview Mockup

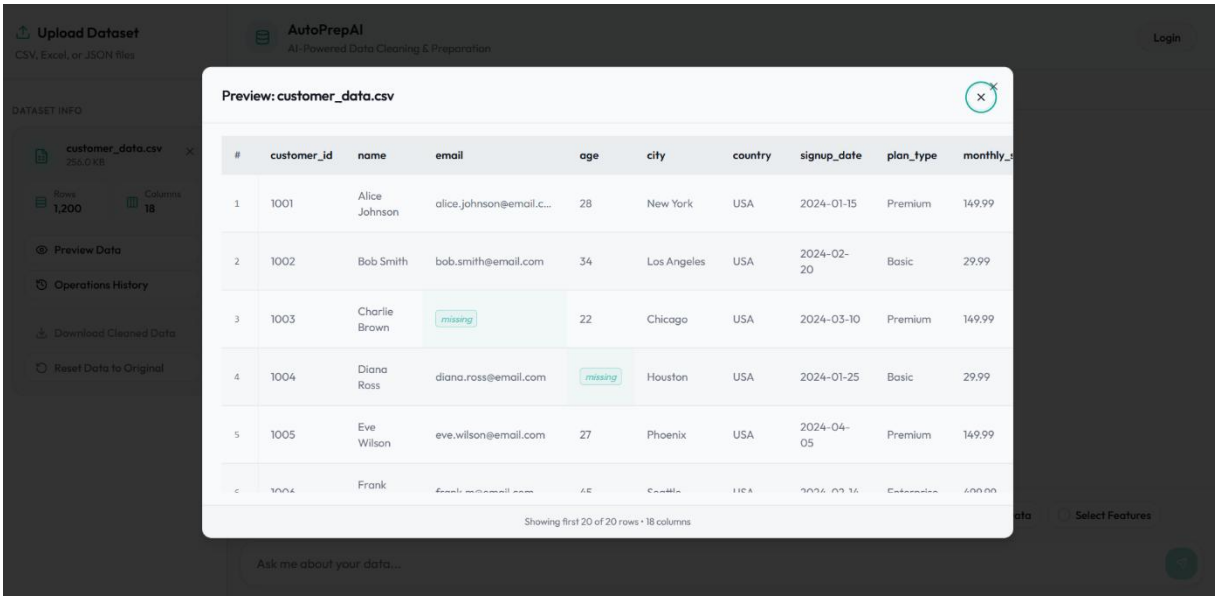


Figure 4-22 Dataset Preview Mockup

Action History Mockup

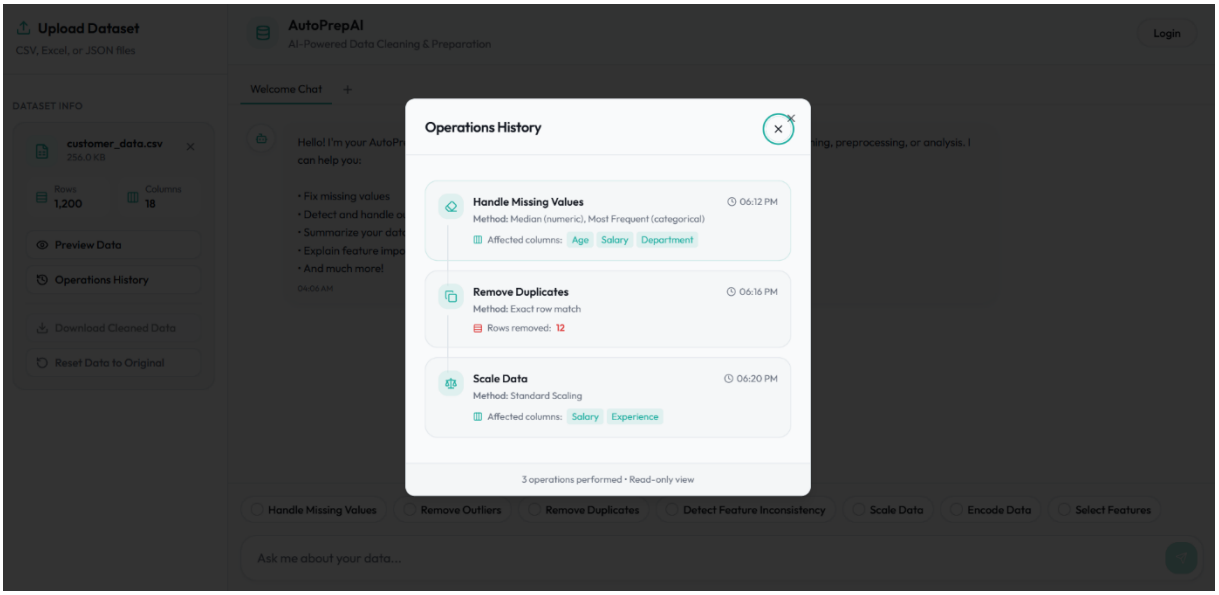


Figure 4-23 Action History Mockup

4.6 Security Design Considerations

Security was considered throughout the design and implementation of the AutoPrepAI backend to protect user accounts, uploaded datasets, and application resources. The system incorporates secure password storage, JWT-based authentication, authorization checks, email verification, input validation, and controlled access to cloud-stored files. These mechanisms work together to ensure that only authenticated users can access protected resources while reducing common security risks such as unauthorized access, credential compromise, and abuse of application services.

4.6.1 Password Security

Passwords are securely stored using the Argon2 password hashing algorithm. During user registration, each password is hashed before being saved to the database, ensuring that plaintext passwords are never stored. During authentication, the submitted password is verified against the stored hash using Argon2's verification mechanism. This approach significantly reduces the risk of credential exposure in the event of a database breach.

The system enforces a password policy requiring a minimum length of eight characters, including at least one uppercase letter, one lowercase letter, one numeric digit, and one special character. This policy encourages users to create stronger passwords that are more resistant to brute-force and dictionary attacks.

During password reset, the system verifies that the new password differs from the user's current password. Preventing password reuse encourages users to select a genuinely new credential, improving overall account security.

4.6.2 Authentication and Session Security

The system uses JSON Web Tokens (JWT) for stateless user authentication. Upon successful login, the server generates a signed JWT containing user identification information and an expiration timestamp. Protected API endpoints require clients to include the JWT in the Authorization header using the Bearer authentication scheme, eliminating the need for server-side session management.

Before granting access to protected resources, the backend validates the token's signature, signing algorithm, and expiration timestamp to ensure that only authentic and unmodified tokens are accepted. It also verifies that the user referenced by the token exists in the database before processing the request. Authentication is implemented as a reusable dependency,

allowing all protected endpoints to apply the same authentication logic consistently while reducing code duplication.

Access tokens remain valid for 60 minutes. Limiting the token lifetime reduces the potential impact of compromised authentication tokens by automatically invalidating them after expiration.

4.6.3 Account Verification and Recovery

Newly created user accounts remain unverified until the email verification process is successfully completed. During registration, the system generates a dedicated email verification token containing the user's email address, an expiration time, and a token type identifier. During verification, the backend validates both the token signature and its intended purpose before activating the user account. Verification tokens automatically expire after one hour to reduce the risk of misuse.

After successful credential verification, the system checks whether the user's email address has been verified. Users with unverified accounts are denied access until they complete the email verification process, ensuring that only users who have proven ownership of their registered email address can access protected resources.

Password recovery uses dedicated JWT tokens that are separate from authentication and email verification tokens. Each reset token contains a unique token type identifier and expires after 15 minutes. Before allowing a password change, the backend validates both the token signature and its type, preventing token reuse across different authentication workflows.

4.6.4 Protection Against Abuse

The password recovery endpoint returns the same response regardless of whether the submitted email address exists in the database. This prevents attackers from determining which email addresses are registered, reducing the risk of user enumeration attacks.

To prevent abuse of the email verification service, the system restricts users from requesting multiple verification emails within a one-minute interval. Requests exceeding this limit are rejected with an appropriate error response, reducing the risk of email flooding and denial-of-service attempts.

To prevent excessive resource consumption, uploaded datasets are validated against predefined size limits before processing. Files exceeding the supported dimensions are rejected, protecting

the application from excessive memory usage and denial-of-service scenarios caused by extremely large uploads.

4.6.5 Authorization and Access Control

While authentication verifies the identity of a user, authorization determines which resources that user is permitted to access. In addition to authenticating users, the backend verifies resource ownership before granting access. For example, conversation retrieval requests are allowed only if the requested conversation belongs to the authenticated user. Unauthorized access attempts result in a 403 Forbidden response, preventing users from accessing other users' data.

Processed datasets are accessed through pre-signed download URLs generated by the backend. Each URL has a limited validity period (one hour), ensuring that stored files cannot be accessed indefinitely and reducing the risk of unauthorized downloads.

4.6.6 Input Validation

The backend validates user input before processing requests. Uploaded datasets, conversation identifiers, and request parameters are checked for validity, and malformed or invalid input is rejected with appropriate HTTP error responses. Early validation reduces the risk of processing invalid data and improves application reliability.

4.6.7 Cross-Origin Protection (CORS)

The backend configures Cross-Origin Resource Sharing (CORS) to allow requests only from trusted frontend origins. Restricting allowed origins helps reduce the risk of unauthorized websites interacting with the API through users' browsers while still allowing legitimate frontend applications to communicate with the backend.

Chapter 5. Implementation

5.1 Implementation Environment and Setup

The AutoPrepAI system was developed using a modern client-server architecture that integrates web technologies, machine learning libraries, and Large Language Models (LLMs). The frontend provides an interactive user interface for dataset upload and preprocessing, while the backend manages AI agents, preprocessing workflows, and database operations.

The implementation environment of the AutoPrepAI system consists of a combination of web development technologies, database tools, cloud services, and AI/ML frameworks. The main components and technologies used during system development are summarized in Table 5-1.

Component	Technology
Programming Language	Python 3.10, JavaScript (ES6)
Frontend Framework	React.js
Backend Framework	FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Cloud Storage	Backblaze B2
AI Framework	DSPy
LLM Interface	LiteLLM
Machine Learning Libraries	Pandas, NumPy, Scikit-learn
Database Driver	Psycopg2
IDE	Visual Studio Code
Version Control	Git & GitHub

Table 5-1 Implementation Environment and Technology Stack

The project requires valid API keys to communicate with cloud-based Large Language Models (LLM). During execution, the backend server is launched using Uvicorn, while the frontend is served through React. PostgreSQL stores user information, uploaded datasets, preprocessing history, and generated recommendations.

5.2 Key Implementation Details (algorithms, modules, major decisions)

The implementation of AutoPrepAI follows modular architecture where each component performs a specific responsibility, improving maintainability and scalability.

5.2.1 Frontend Module

The frontend was implemented using React.js to provide an interactive and responsive interface. Users can:

- Register and log into the system.
- Upload CSV datasets.
- Preview dataset contents.
- View logs and actions.
- Accept or reject each action taken.
- Monitor preprocessing progress.
- Download the cleaned dataset.

The interface emphasizes simplicity while supporting Human-in-the-Loop interaction during preprocessing.

5.2.2 Backend Module

The backend was developed using FastAPI, which exposes RESTful APIs responsible for:

- User authentication.
- Dataset upload.
- AI recommendation generation.
- Dataset preprocessing.
- Communication with the frontend.
- Database management.

FastAPI was selected because of its high performance, automatic API documentation, and seamless integration with Python machine learning libraries.

5.2.3 Human-in-the-Loop Mechanism

One of the main contributions of AutoPrepAI is the Human-in-the-Loop (HITL) workflow. Rather than executing preprocessing automatically, the system pauses after generating recommendations and requests user approval. Users may:

- Accept / Reject the action taken

This approach combines AI automation with human expertise, improving transparency and user trust.

5.2.4 Data Preprocessing Pipeline

The preprocessing pipeline executes the approved operations sequentially.

Typical preprocessing operations include:

- Missing value imputation
- Duplicate removal
- Outlier treatment
- Feature scaling
- Categorical encoding
- Data type conversion

Each preprocessing step updates the dataset before passing it to the next stage.

5.2.5 Database and Cloud Storage Module

The system uses PostgreSQL as its primary relational database to store structured application data, including:

- User accounts and profiles
- Authentication information
- Dataset metadata
- Preprocessing sessions
- AI-generated actions
- Processing history and user decisions

SQLAlchemy ORM is used to manage communication between the FastAPI backend and the PostgreSQL database, simplifying database operations and improving maintainability.

To efficiently manage uploaded datasets, **Backblaze B2 Cloud Storage** is integrated into the system. Instead of storing large CSV files directly in the database, uploaded datasets are securely stored in the cloud, while PostgreSQL maintains references and metadata associated with each file. This approach reduces database storage requirements, improves scalability, and enables reliable access to datasets throughout the preprocessing workflow.

5.2.6 Major Implementation Decisions

- **Choice of FastAPI:** FastAPI was selected because of its high performance, automatic API documentation, and seamless integration with Python-based machine learning libraries.

- **Choice of React:** React was selected to create a responsive and interactive user interface while allowing reusable components and efficient state management.
- **Choice of PostgreSQL:** PostgreSQL was chosen because it provides reliable relational data management, supports complex queries, and integrates well with SQLAlchemy.
- **Choice of Backblaze B2:** Backblaze B2 Cloud Storage was selected to store uploaded datasets efficiently while reducing database storage requirements and improving scalability.
- **Choice of Human-in-the-Loop:** Instead of fully automating preprocessing, the system keeps users involved in the decision-making process. This improves transparency, increases user trust, and allows domain experts to validate AI-generated recommendations before execution.

5.3 Sample Screenshots / Output Samples

Signup page

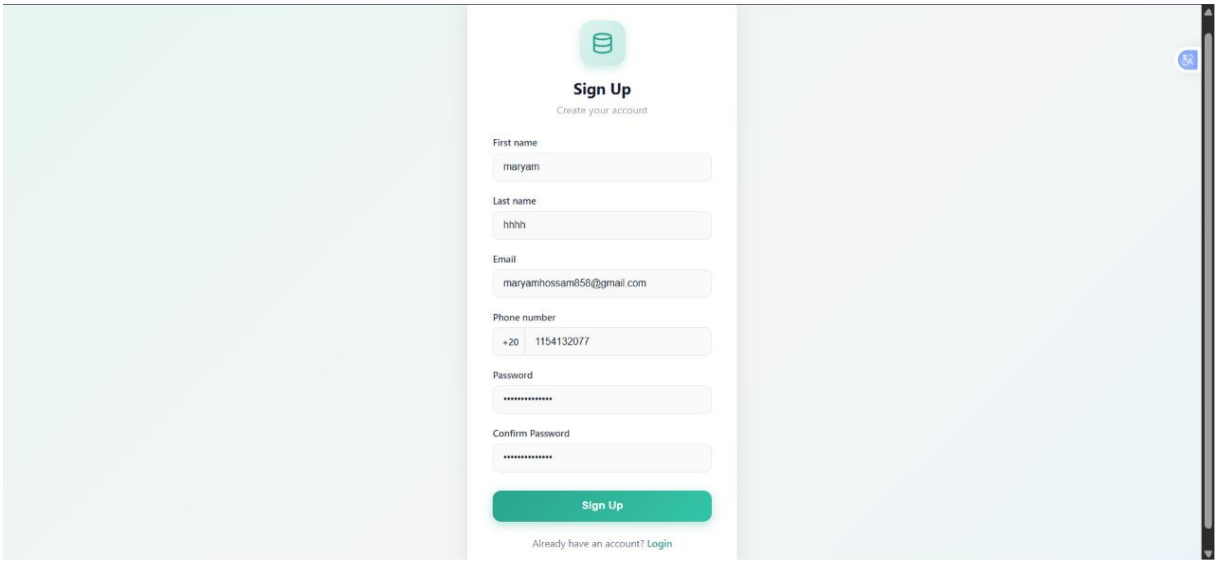


Figure 5-1 User Registration Interface (Sign-Up Page)

Verifying used mail in sign up exists

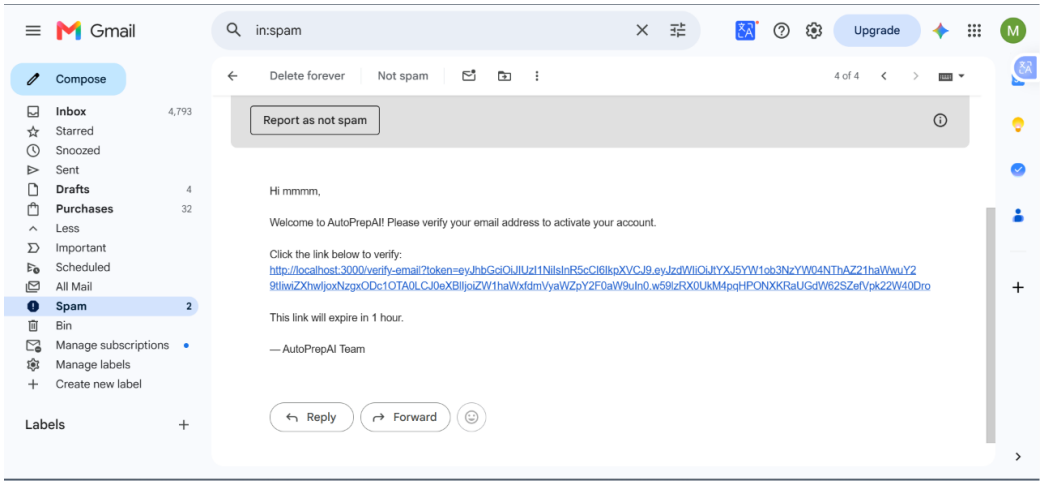


Figure 5-2 Email Verification During User Registration

Login page

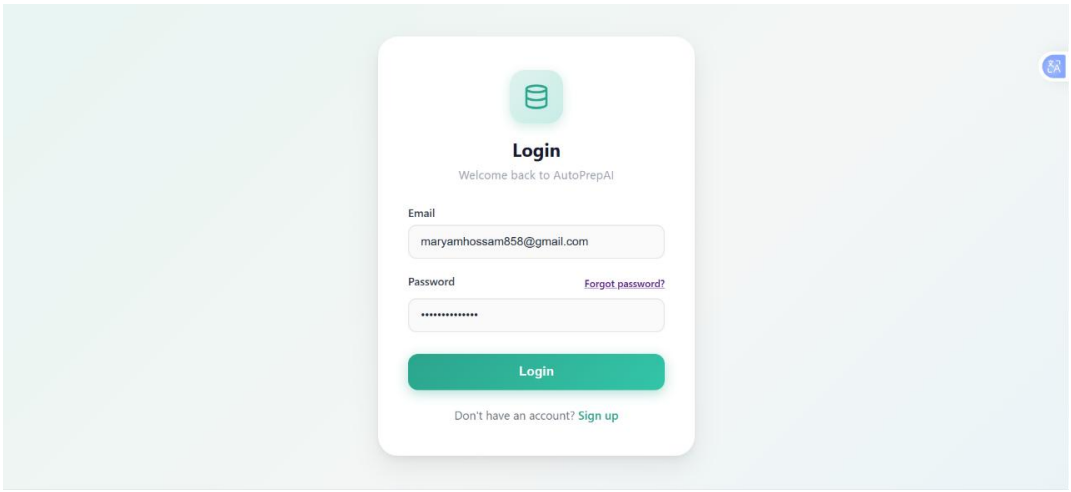


Figure 5-3 User Authentication Interface (Login Page)

Reset link in case of forget password

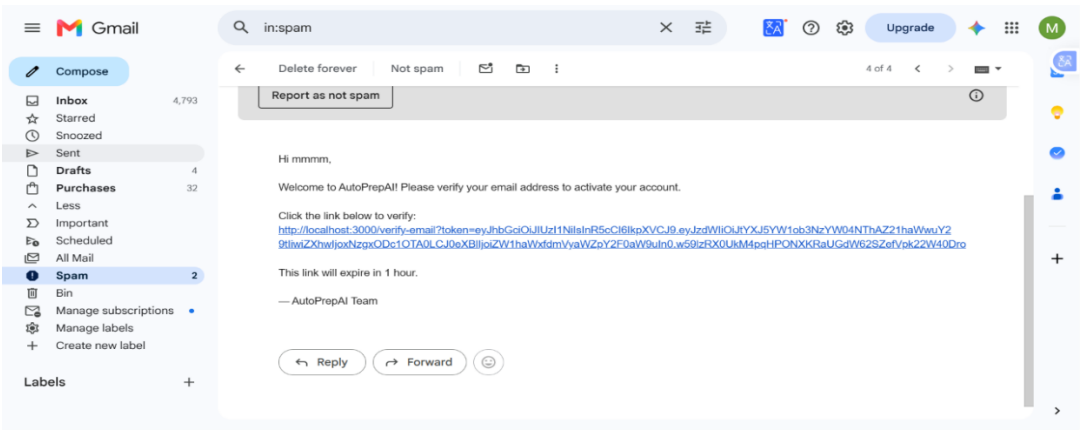


Figure 5-4 Password Recovery and Reset Link Functionality

Home page

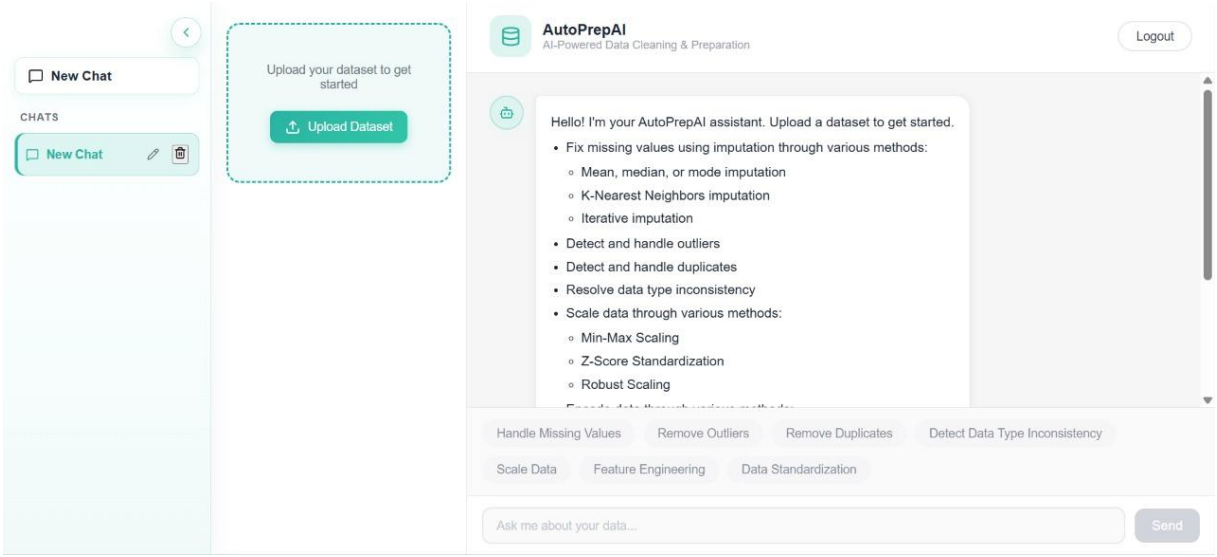


Figure 5-5 AutoPrepAI Home Dashboard

Data uploaded

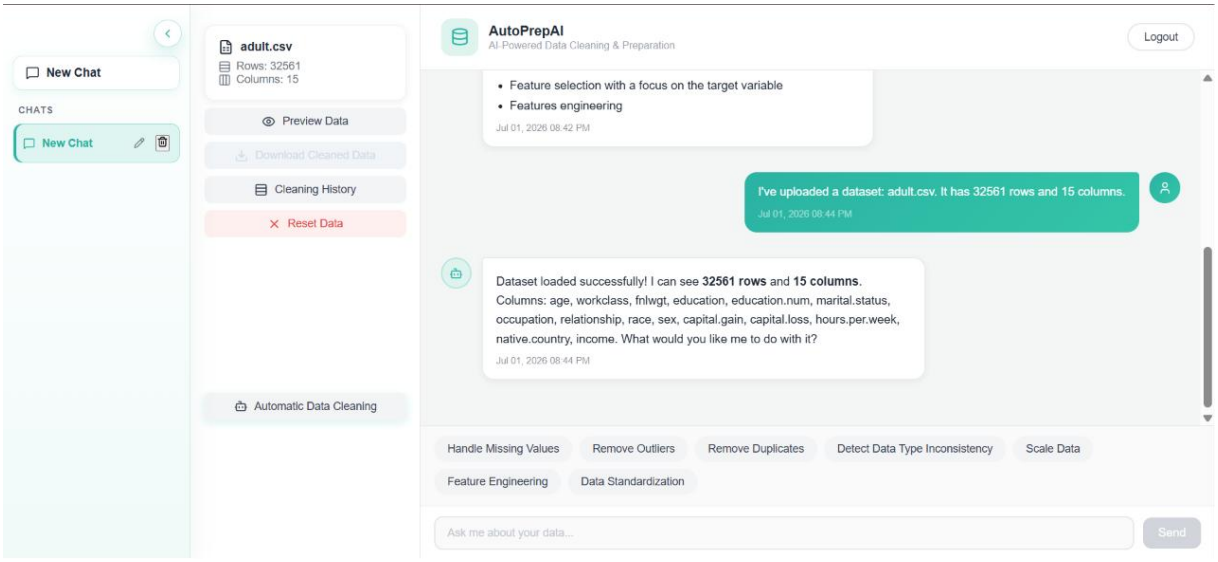
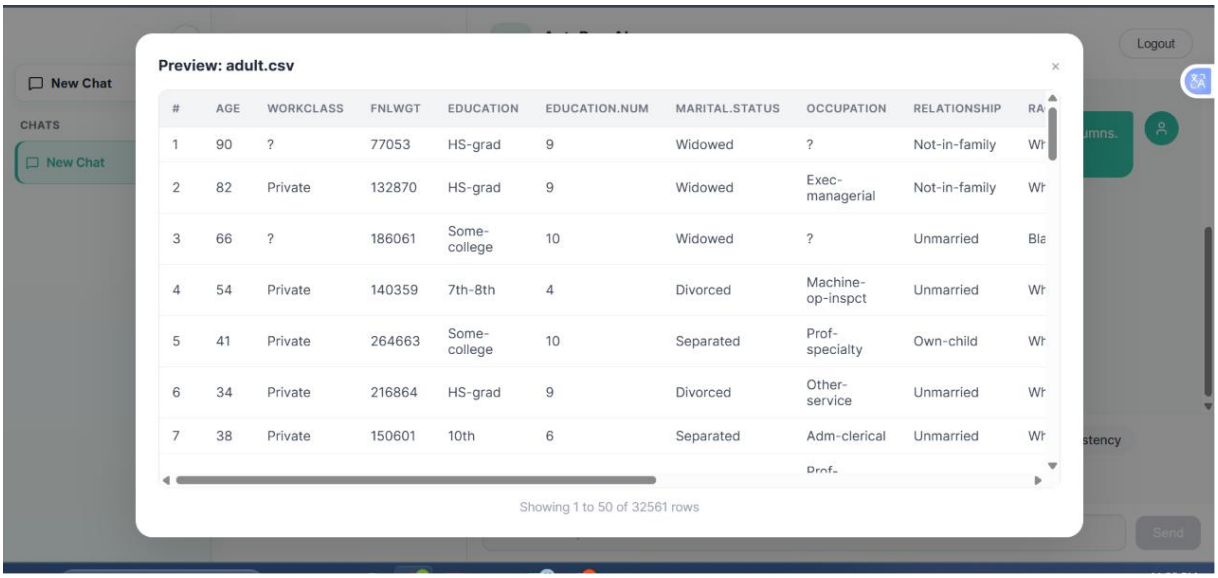


Figure 5-6 Dataset Upload Interface

File preview for data



#	AGE	WORKCLASS	FNLWGT	EDUCATION	EDUCATION.NUM	MARITAL.STATUS	OCCUPATION	RELATIONSHIP	RA
1	90	?	77053	HS-grad	9	Widowed	?	Not-in-family	Wh
2	82	Private	132870	HS-grad	9	Widowed	Exec-managerial	Not-in-family	Wh
3	66	?	186061	Some-college	10	Widowed	?	Unmarried	Bla
4	54	Private	140359	7th-8th	4	Divorced	Machine-op-inspct	Unmarried	Wh
5	41	Private	264663	Some-college	10	Separated	Prof-specialty	Own-child	Wh
6	34	Private	216864	HS-grad	9	Divorced	Other-service	Unmarried	Wh
7	38	Private	150601	10th	6	Separated	Adm-clerical	Unmarried	Wh

Figure 5-7 Raw Dataset Visualization Before Cleaning

Sample input using NLP and Quick actions

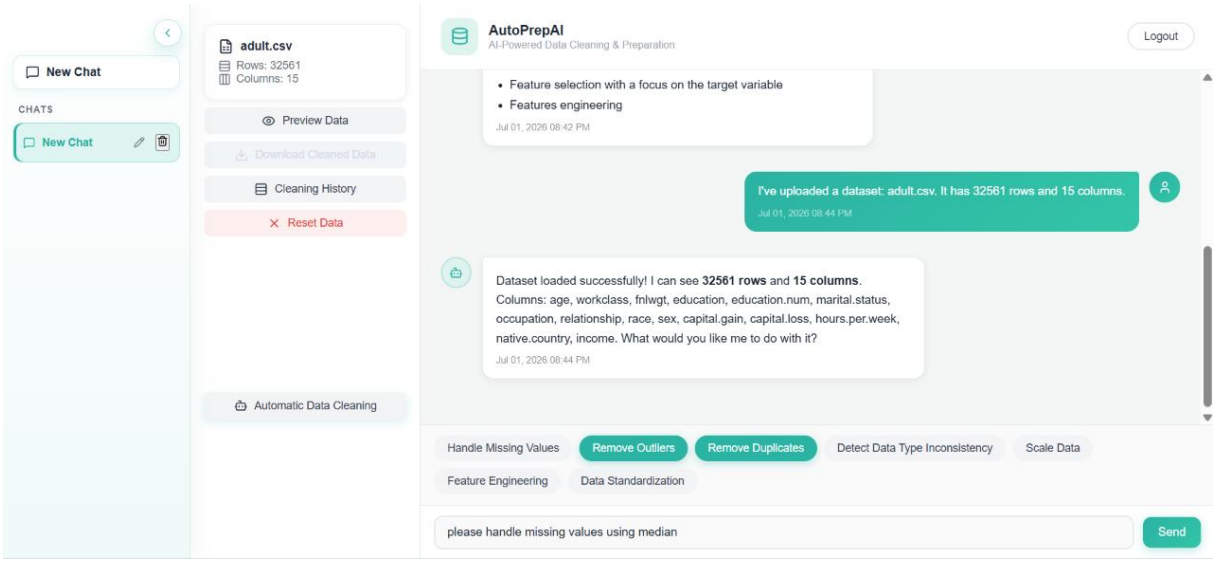


Figure 5-8 Natural Language Input and Quick Action Selection

Start of pipeline

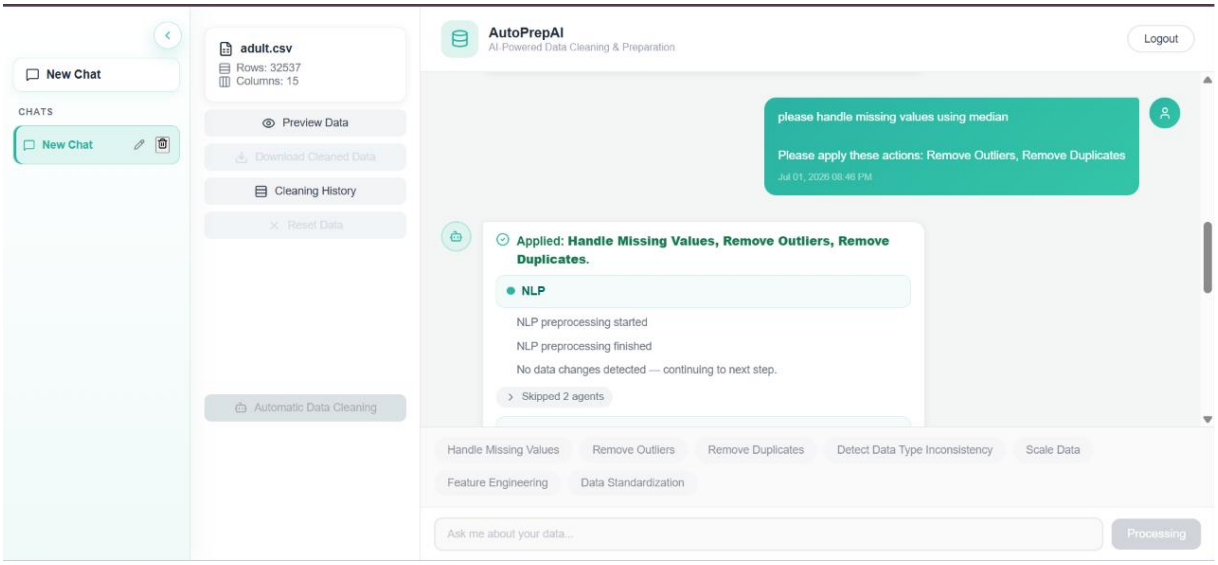


Figure 5-9 Initialization of the Data Cleaning Pipeline

Duplicates handling processing and explanation

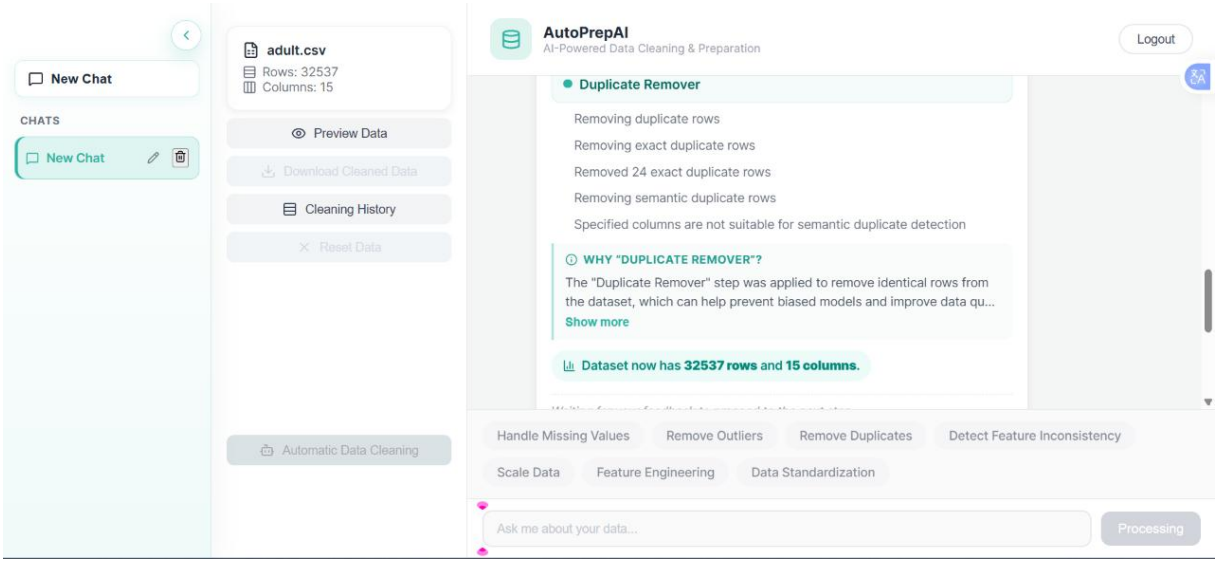


Figure 5-10 Duplicate Detection and Removal Process with Explanation

Sample of human in loop happening after each action

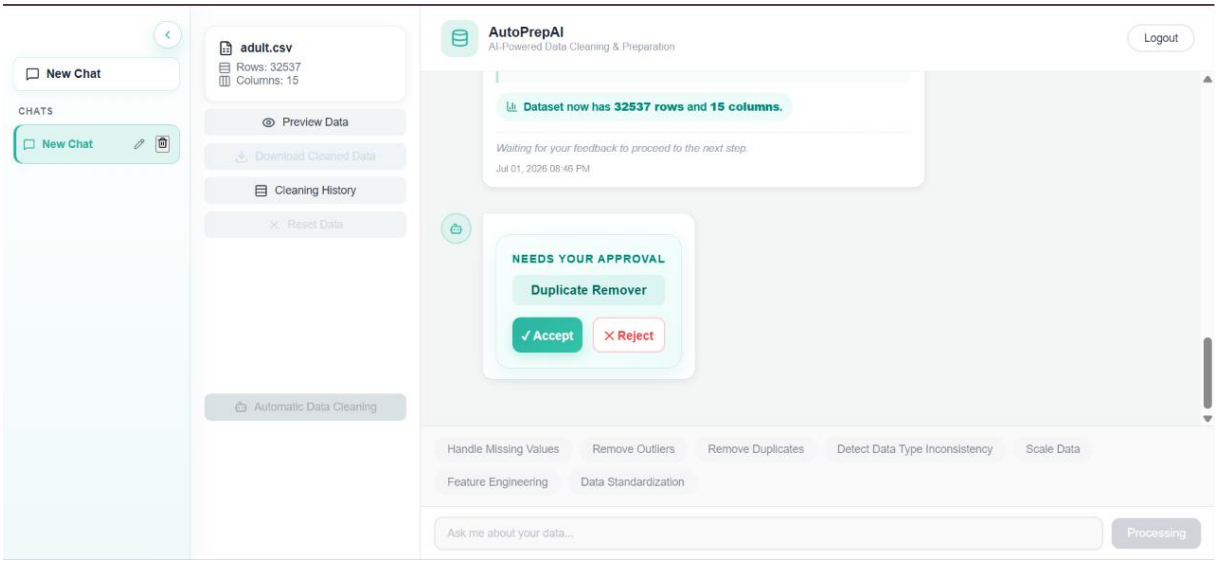


Figure 5-11 Human-in-the-Loop Validation During Processing

Outliers handling processing and explanation

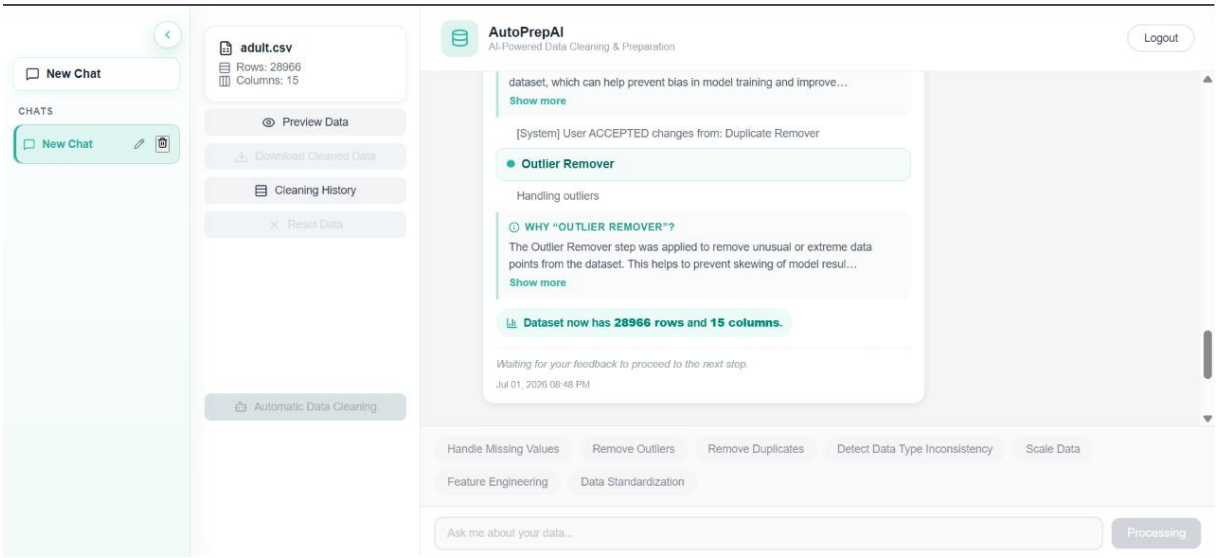


Figure 5-12 Outlier Detection and Handling Process with Explanation

Missing values handling processing and explanation

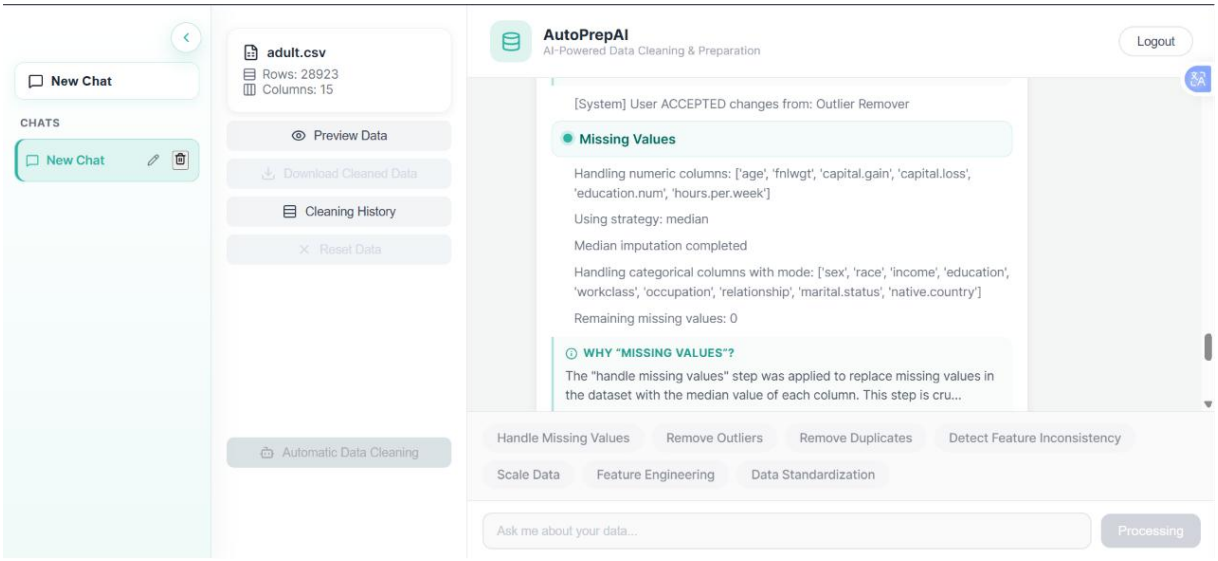


Figure 5-13 Missing Value Detection and Imputation Process with Explanation

End of processing pipeline successfully

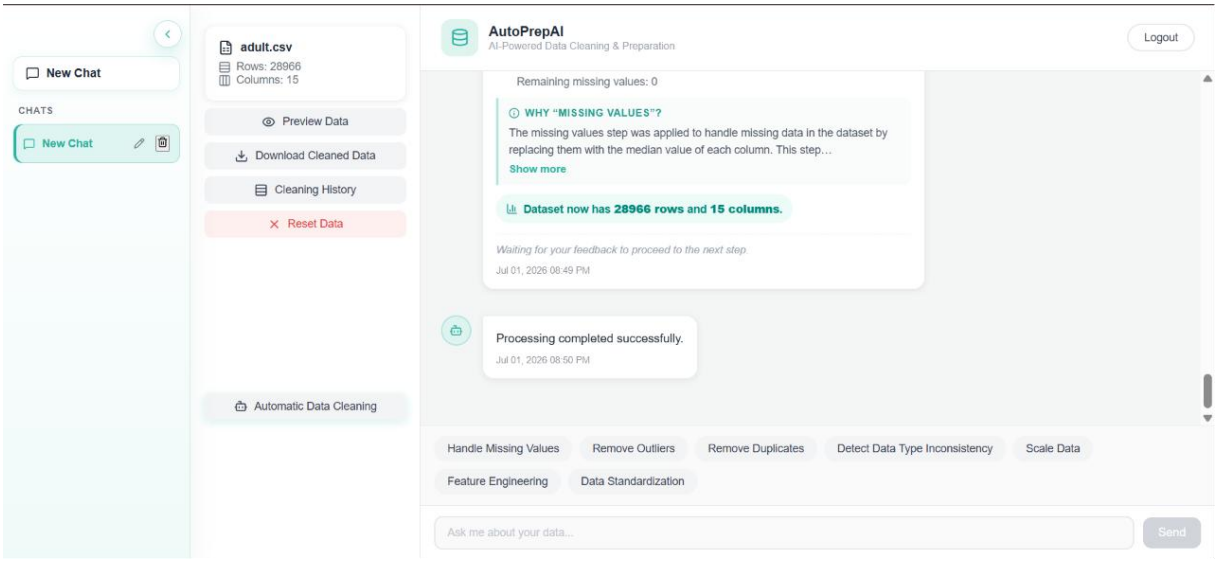


Figure 5-14 Successful Completion of the Data Processing Pipeline

Output (Data after cleaning)

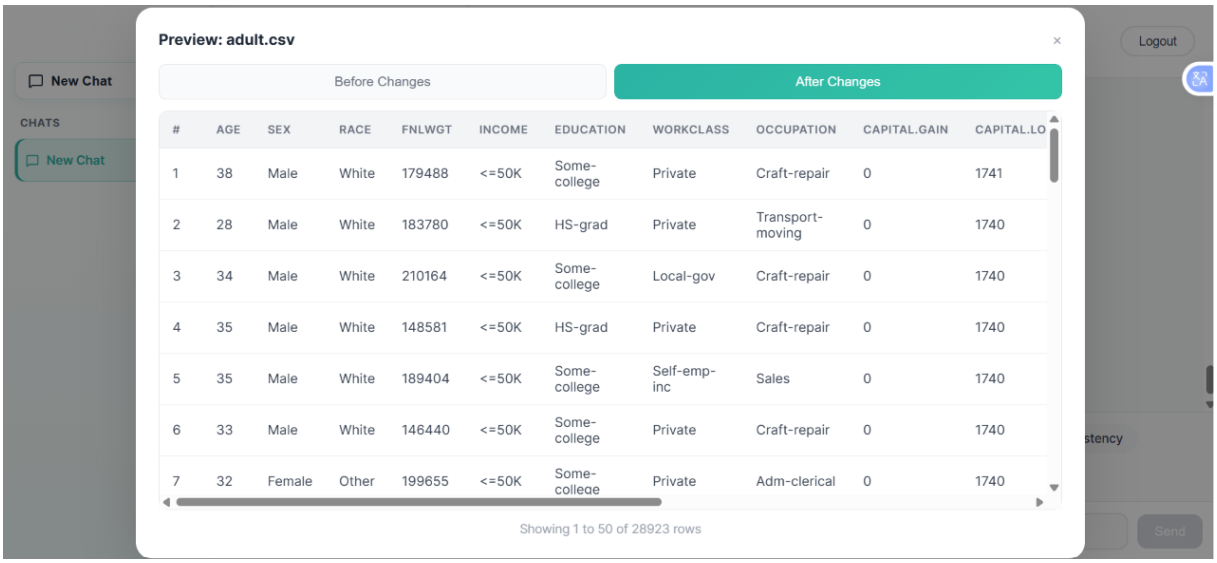


Figure 5-15 Cleaned Dataset Output After Processing

Sample of operation history

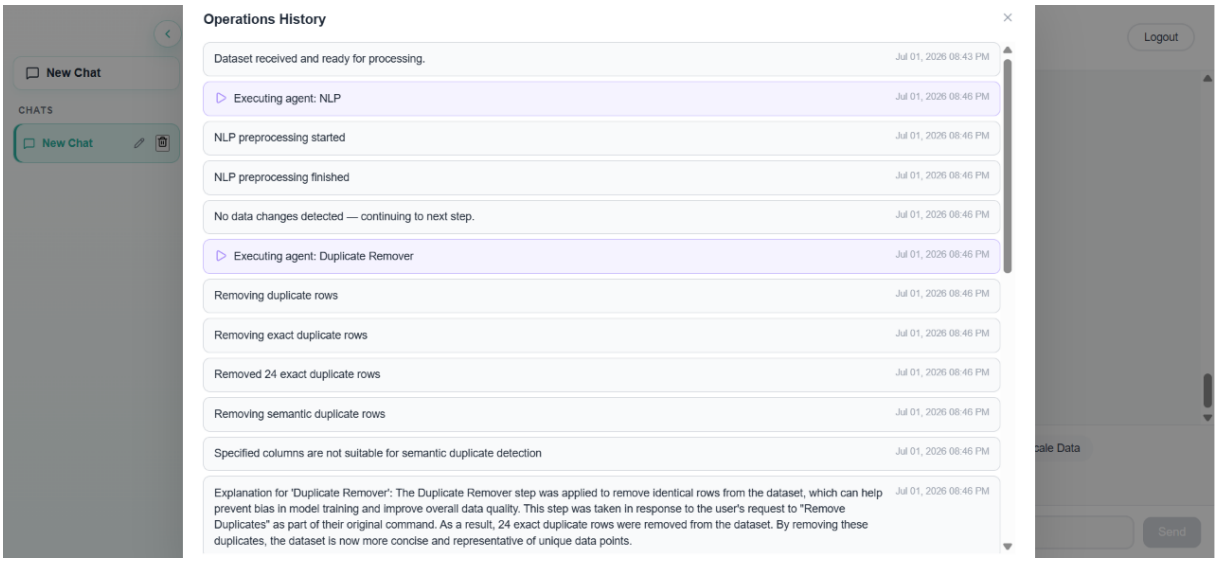


Figure 5-16 Operation History and Processing Log Interface

Chapter 6. Testing and Evaluation

6.1 Testing Strategy

The testing strategy for AutoPrepAI was designed to evaluate the system at both the individual agent level and as an integrated, end-to-end pipeline.

- Agent-Level Testing: Each of the six intelligent agents was evaluated independently using targeted datasets (e.g., Titanic [11], WDC Product Matching [12], UCI Adult [13], Credit Card Fraud [14]) to isolate and measure specific functionalities like anomaly detection, semantic duplicate recall, and feature inconsistency resolution.
- End-to-End Pipeline Testing: The integrated framework was tested on the UCI Adult [13] and Diabetes 130-US Hospitals [15] datasets using a downstream Random Forest classifier. To ensure robustness and prevent data leakage, the evaluation utilized 30 repeated stratified splits, fitting the preprocessing pipeline strictly on the training splits.
- Human-Centric Validation: A practitioner feedback survey involving 32 machine learning and data science professionals was conducted to evaluate the system's usability, explainability, and interaction design.
- Software Unit and Integration Testing: To ensure the reliability and proper functioning of the backend architecture, we implemented a complete set of automated unit and integration tests. We tested individual modules and services, including the intelligent preprocessing agents, cloud storage interactions, and encoding logic, in strict isolation. Additionally, we used broader integration tests to check the smooth coordination between the pipeline builder components. This explicitly verified the correct separation of data standardization and inconsistency resolution in both manual and automatic execution modes.

6.2 Test Cases and Results

The following test cases were conducted to evaluate the performance of the proposed system components across different datasets. The evaluation includes preprocessing agents, performance metrics, and user reception results, as summarized in Table 6-1.

Agent / Module	Dataset	Test Case / Metric	Results / Performance
Outlier Filtering	Titanic [11]	Evaluate accuracy using Isolation Forest vs. Z-Score/IQR	Achieved **0.722** accuracy (highest among strategies), outperforming Z-Score (0.712) and IQR (0.685).

Outlier Filtering	Credit Card Fraud [14]	Evaluate ROC-AUC for high-dimensional ($d > 4$) skewed data	Achieved **0.743** ROC-AUC. Effectively isolated multi-dimensional anomalies, significantly outperforming Z-Score (0.596).
Duplicate Detection	WDC Product Matching (Computers & Cameras) [12]	Recall & F1 via SBERT multi-column embeddings vs. Exact Match	**Computers:** 86.0% Recall (258/300 found) with 0.485 F1, a 20× recall gain over exact matching (4.3%). **Cameras:** 74.0% Recall with 0.556 F1.
Missing Value Imputation	Titanic [11]	Adaptive strategy selection via RMSE on masked samples (MAR context)	Automatically selected KNN, achieving lowest MAE (**6.75** vs median's 12.21) and improving downstream model F1 to **0.763**.
Inconsistency Resolution	UCI Adult [13]	Precision & Recall of LLM mapping with safety validation gates ($T = 0$)	**100% Precision** (0 false positives) and **87.5% Recall** (F1=0.933). The safety gates successfully auto-rejected 2 anomalous LLM proposals to prevent hallucinations.
Feature Selection	Titanic [11]	F1 Score using RF Importance vs. Brute-Force Baseline	Achieved **0.782** F1 Score (up from 0.768 baseline), effectively reducing dataset dimensionality by 50% (from 10 features to 5).
Feature Engineering	Titanic [11]	F1 Score using Selective Acceptance via DSPy vs. Accepting All	Achieved **0.782** F1 Score. Validation layer retained 2/5 valid generated expressions, preventing the overfitting seen when accepting all 5 features (0.769 F1).
End-to-End Pipeline	Diabetes 130-US Hospitals [15]	30-run repeated stratified downstream RF classification	Statistically significant improvement ($p < 0.001$), achieving **0.6016** Accuracy and **0.5549** weighted F1, beating raw and fixed-rule baselines.
User Reception	Survey ($n = 32$)	Perceived usefulness, interaction preferences, and explainability	**87.5%** rated the system Useful/Very Useful; **93.8%** rated explainability as Critical/Important; **84.4%** preferred the flag-and-confirm HITL duplicate approach.

Table 6-1 Test Cases and Evaluation Results

6.3 Performance Evaluation / Benchmarking (where applicable)

To evaluate the effectiveness of AutoPrepAI, we benchmarked its end-to-end performance against two baseline approaches. A Raw-minimal setup (basic encoding and train-test split, no

real preprocessing logic). A Fixed-rule setup (deterministic preprocessing steps, no adaptive decision making).

- Diabetes 130-US Hospitals Dataset [15]: Here, AutoPrepAI clearly outperformed both baselines and the improvement was significant ($p < 0.001$). It achieved an accuracy of 0.6016 and a weighted F1 score of 0.5549 confirming that its adaptive preprocessing pipeline really adds value on this dataset.
- UCI Adult Dataset [13]: Results were a bit more nuanced. AutoPrepAI beat the fixed-rule baseline (0.8514 vs. 0.8363 accuracy), but came in slightly below the raw-minimal baseline. This isn't necessarily a weakness, it actually reinforces a key insight: AutoPrepAI's adaptive approach shines most on noisier, messier data, where there's more for it to actually fix. On a relatively clean dataset like Adult, there's simply less room for adaptive preprocessing to add value, but importantly, it still didn't compromise data integrity in the process.

6.4 Limitations and Known Issues

Despite the system performing well overall, the evaluation highlighted the following limitations:

- Semantic Deduplication Precision: The SBERT-based semantic duplicate detection achieves exceptional recall but moderate precision (e.g., an F1 score of 0.485 on the WDC Computers dataset [12]). False positives occasionally occur with same-brand product variants, necessitating the human-in-the-loop (HITL) confirmation step to prevent silent data loss.
- Cost and Latency Overheads: The reliance on cloud-based Large Language Models (LLMs) for inconsistency resolution and feature engineering introduces API latency, per-query financial costs, and a dependency on external services.
- Dataset-Dependent Efficacy: End-to-end pipeline gains are heavily dependent on the nature of the dataset, showing the most significant performance boosts on highly noisy data rather than cleanly structured data.
- Explanation Evaluation: While practitioners confirmed a critical need for explainability, the subjective quality and trust calibration of the LLM-generated explanations have not yet been evaluated through controlled human-computer interaction studies.

Chapter 7. Conclusion and Future Work

7.1 Summary of Achievements

The AutoPrepAI project successfully bridged the gap between automated data preprocessing and transparent data engineering. Key achievements include:

- Unified Multi-Agent Architecture: Developed an integrated system featuring six intelligent agents capable of handling missing values, duplicates, outliers, inconsistencies, and feature engineering.
- Semantic Duplicate Detection: Engineered a hybrid detection method using SBERT embeddings and Approximate Nearest Neighbor (ANN) search, achieving a 20x higher recall rate than standard exact-matching techniques.
- Explainable AI Integration: Implemented operation-level justifications and detailed audit logs, addressing the "black box" nature of enterprise AutoML tools.
- Strong Industry Validation: Secured highly positive feedback from industry practitioners, with 93.8% confirming the critical importance of the system's explainability features.

7.2 Lessons Learned

We learned some important lessons in development and evaluation:

- Human-in-the-Loop Required: Pure automation is too dangerous for destructive operations such as dropping rows. The survey reiterated that 84.4% of practitioners prefer flag-and-confirm to auto deletion.
- Trust is Built by Explainability: Providing plain-English justifications for why a specific preprocessing strategy was chosen (e.g. selecting median over mean because of skewness) directly addresses a major pain point, as almost 60% of surveyed practitioners reported having to previously undo automated changes they couldn’t understand.
- Domain Context Required for Adaptive Strategy Choice: The varying performance across datasets (e.g., big wins on Diabetes but mixed results on Adult) indicates that generic decision rules are insufficient; systems must be able to learn or be informed about data properties to make effective preprocessing choices.
- Cost Management Needs Adoption: LLM-powered features (semantic duplicate detection, inconsistency resolution) have significant value, but API costs and latency

concerns prevent enterprise deployment, leading to hybrid approaches and local inference options.

7.3 Future Enhancements

To build upon the current architecture, future iterations of AutoPrepAI should focus on the following enhancements:

- Multimodal and Unstructured Data Support: We are moving beyond purely structured tabular datasets. We will expand the system's ability to parse and reason with unstructured text, JSON objects, and possibly multimodal data streams.
- Active Learning in the Human-in-the-Loop (HITL) Layer: This involves creating a feedback loop. The decision engine learns from the user's manual approvals and rejections, especially within the Duplicate Detection and Inconsistency modules. Over time, this will help the system adjust its internal thresholds and lower the manual validation workload.
- Improved Visual Explainability: Upgrading the frontend interface to work alongside the text-based LLM justifications with interactive visual diagnostics. This includes overlaying pre- and post-imputation data distributions and visualizing how feature importance changes after removing outliers.
- Cross-Domain Generalization Testing: Testing AutoPrepAI on a wider variety of datasets, such as financial, healthcare, and IoT sensor data, helps identify where its strengths remain and where they fail. This process strengthens the claims about the generalizability of the framework.
- Multilingual NLP Capabilities: We will improve the current English-only Natural Language Processing (NLP) pipeline. This includes adding multilingual embedding models, like multilingual Sentence-BERT, and large language models (LLMs) to process and standardize datasets in different languages.
- Distributed Processing for Big Data: To tackle the current limit of 5,000,000 cells, we will upgrade the data processing engine to enable distributed computing frameworks, like Apache Spark or Dask. This upgrade will allow for out-of-core processing and parallel execution with large datasets.

- Enterprise Collaboration and Organizational Workspaces: We will shift the platform from a single-user tool to a collaborative data engineering environment. This improvement will introduce "Organizational Workspaces," where multiple data engineers can securely share datasets, work on preprocessing pipelines together, and track changes. It will feature Role-Based Access Control (RBAC), real-time synchronization, and version control for data preprocessing states.

References

[1] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning, Cambridge, MA: MIT Press, 2016.

[2] J. Han, M. Kamber and J. Pei, Data Mining: Concepts and Techniques, Waltham, MA: Morgan Kaufmann, 2012.

[3] P. J. García-Laencina, J. L. Sancho-Gómez and A. R. Figueiras-Vidal, "Pattern classification with missing data: A review," *Neural Computing and Applications,* 2010.

[4] H. Touvron, "Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023.

[5] OpenAI, "GPT-4 Technical Report," 2023.

[6] DataRobot, "DataRobot: Enterprise AI Cloud Platform," [Online]. Available: https://www.datarobot.com/.

[7] H2O.ai, "H2O Driverless AI: Automatic Machine Learning Platform," [Online]. Available: https://h2o.ai/.

[8] Trifacta, "Trifacta Data Wrangling Platform," [Online]. Available: https://www.trifacta.com/.

[9] Alteryx, "Alteryx Analytics Platform," [Online]. Available: https://www.alteryx.com/.

[10] Numerous.ai, "AI Assistant for Spreadsheets," [Online]. Available: https://numerous.ai/.

[11] Kaggle, "Titanic: Machine Learning from Disaster".

[12] A. Primpeli, R. Peeters and C. Bizer, The WDC Training Dataset and Gold Standard for Large-Scale Web-Specific Product Matching, 2019.

[13] D. Dua and C. Graff, "UCI Machine Learning Repository," University of California, Irvine, School of Information and Computer Sciences, 2019.

[14] A. Dal Pozzolo, O. Caelen, R. A. Johnson and G. Bontempi, "Calibrating Probability with Undersampling for Unbalanced Classification," in *IEEE Symposium Series on Computational Intelligence*, 2015.

[15] B. Strack and e. al., "Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records," *BioMed Research International,* 2014.